# Database Management Capstone Project Report

# Hospital Management System

*Submitted in partial fulfilment of the requirement of the degree of*

**BACHELOR OF TECHNOLOGY**

**in**

**CSE with Specialization Data Science (Section -A)**

**to**

# K.R Mangalam University

**by**


## Abhinav Kumar Singh (2501420071)


**Course: Database Management System (ETCCMS304)**

**Submission Date: 10 November 2026**

Under the supervision of

## Dr. Swati Gupta


Department of Computer Science and Engineering, School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001 India, November 2026

# INDEX


| S.No. | Topic of Content | Page No. |
|---|---|---|
| 1 | Abstract | 3 |
| 2 | Introduction | 4 |
| 3 | Objectives | 5 |
| 4 | Problem Statement | 6 |
| 5 | Database Design | 7-9 |
| 6 | Tools and Platforms Used | 10 |
| 7 | SQL Implementation and Queries | 11-56 |
| 8 | Indexing and Query Optimization | 57-60 |
| 9 | Views and Access Control | 61-63 |
| 10 | Window Functions | 64-67 |
| 11 | Results and Discussion | 68 |
| 12 | Conclusion | 69 |
| 13 | References | 70 |
| 14 | Appendix A: SQL Execution Outputs | 71-73 |

# ABSTRACT

This project develops a relational database for a Hospital Management System named HospitalDB. The database is intended to organize the information that a hospital uses during routine operations, including departments, doctors, patients, appointments, beds, admissions, medical history, treatments and surgeries. Instead of keeping these records in unrelated files, the project stores them in connected tables. Each table has a primary key to identify its records, while foreign keys connect related information and help prevent references to records that do not exist. The implementation uses SQL to create tables, insert sample records and retrieve operational reports. Queries are used to identify current in-patients, review appointment status, locate available beds, display surgery schedules and summarize department-wise admissions. The project also demonstrates database features beyond basic data entry: composite indexes are introduced for frequently used searches, views present selected information to different users, and window functions support ranking and comparisons. The sample data is small and is used to demonstrate the design and query logic; it is not a production hospital dataset. The project therefore provides a learning model of how a structured database can support accurate record keeping, controlled access and more organized hospital operations.

The sample records make the database behaviour easier to verify. For example, the admission query identifies rows whose discharge_date is NULL, while the bed query reads the availability value stored in Bed. These are related but separate checks: the current design does not automatically update a bed's status when an admission is inserted or discharged. This distinction is important when interpreting the sample output and when planning a production-ready system.

# INTRODUCTION

A hospital generates and updates information throughout the day. A patient may register, book an appointment, consult a doctor, receive a diagnosis, undergo treatment and, if required, be admitted to a bed or scheduled for surgery. These activities are connected: an appointment must refer to a real patient and doctor, an admission must refer to a real patient and bed, and a surgery must refer to the patient, doctor and department involved. If these details are maintained in separate paper files or disconnected spreadsheets, staff may need to search several places to answer a simple question. The same information may also be entered more than once, creating a risk of spelling differences, outdated phone numbers or conflicting records. A relational database addresses this problem by storing each main type of information in a suitable table and connecting the tables through keys. In this project, HospitalDB contains nine tables: Department, Doctor, Patients, Appointment, Bed, Admission, Medical_History, Treatment and Surgery. The structure separates descriptive information from activity records. For example, a patient's name is stored once in Patients, while Appointment stores the patient_id and doctor_id needed to connect a visit to the correct people. SQL makes it possible to retrieve combined information through joins, summarize records through aggregate functions and filter the results according to operational needs. The project also introduces indexes to support frequent searches, views to limit the columns exposed to front-desk staff and window functions to compare records without collapsing individual rows. The design is a database foundation rather than a complete hospital application: screens, authentication workflows, billing and real-time integrations would be separate future development work.

The schema separates master data from transaction data. Department, Doctor, Patients and Bed describe the main entities, whereas Appointment, Admission, Medical_History, Treatment and Surgery record activities involving those entities. This separation reduces repeated descriptive data and allows the same patient or doctor to appear in many activity records without copying their name and contact details into every row.

The report follows the database development process in sequence: define the schema, load representative records, retrieve information with SQL, and then apply indexes, views and window functions. Each query is paired with its purpose and an explanation of the logic so that the output can be interpreted rather than merely copied. The final discussion also records the limitations of the sample design and identifies practical improvements for a larger deployment.

# OBJECTIVES

The main objectives of this project are:

1. To design a relational database schema for a hospital that stores patient records, medical history, appointments and treatments.
2. To manage the information of departments, doctors, beds, admissions and surgeries.
3. To use primary keys, foreign keys and NOT NULL constraints to maintain data integrity.
4. To track patient admissions and discharges using the admission and discharge dates.
5. To write SQL queries using joins, grouping, aggregate functions and subqueries for hospital reports.
6. To support hospital resource management by finding available beds and department-wise admissions.
7. To schedule and display surgeries with the patient, doctor and date.
8. To improve query performance using indexes and to analyse query plans using EXPLAIN.
9. To create views for current in-patients and a restricted front-desk view that hides medical details.
10. To use window functions DENSE_RANK(), AVG() OVER(), LAG() and LEAD() for analysis of hospital data.

# PROBLEM STATEMENT

Hospitals need reliable access to connected information about patients, staff, appointments, admissions, beds, treatments and surgeries. The central problem addressed by this project is how to store these records so that they can be retrieved consistently and combined when staff need operational information. A poorly organized system can make it difficult to identify a patient's previous diagnosis, confirm whether an admission is still active, find an available bed or prepare a doctor-wise appointment list. The database solution in this report uses a relational design. Separate tables represent the main entities and foreign keys connect the records. SQL queries then combine these tables to answer specific questions. Indexes are included for selected search patterns, while views provide a narrower presentation of information for users who do not need access to every medical detail. The project demonstrates these techniques with a limited sample dataset and explains what the results mean.

**Problems Being Faced**

1. Scattered Patient Records: Medical history, treatment and appointment details are difficult to retrieve when they are not stored in connected tables.
2. Bed and Resource Management: Without a record of bed availability and admissions, it is hard to know which beds are free.
3. Admission and Discharge Tracking: Hospitals need to know which patients are still admitted and which have been discharged.
4. Surgery Scheduling: Surgeries must be scheduled with the correct patient, doctor and department.
5. Data Privacy: Front-desk staff should see appointment details but not the diagnosis or treatment of patients.
6. Slow Queries: As the data grows, searching the appointments or admissions without indexes becomes slow.

**Proposed Solution**

A relational Hospital Management Database is proposed in MySQL. Tables with primary and foreign keys keep the data consistent, SQL queries generate the required reports, indexes speed up searching, views provide secure access to selected data, and window functions allow ranking and comparison of records.

# DATABASE DESIGN

## 1.1 Database Overview

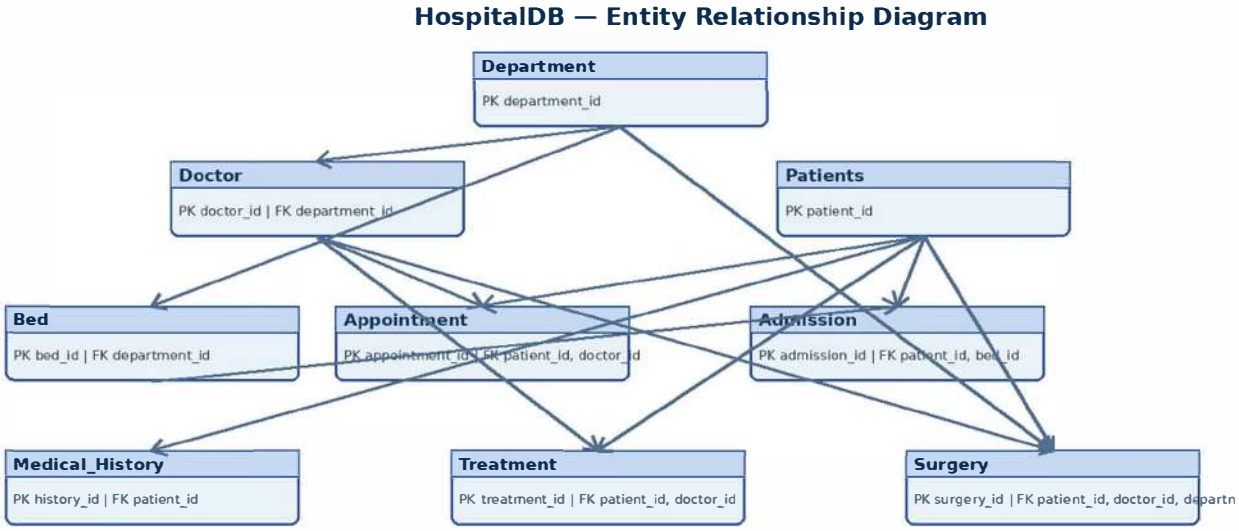
HospitalDB is organized around nine related tables. The design separates relatively stable details, such as a department's name or a patient's date of birth, from events that happen over time, such as appointments, admissions, treatments and surgeries. This separation reduces unnecessary repetition and makes it easier to update a record in one place. Department is a parent table for Doctor, Bed and Surgery. Patients is the central patient-information table and is referenced by Appointment, Admission, Medical_History, Treatment and Surgery. Appointment links patients and doctors, while Admission links a patient to the bed assigned for a stay. The foreign-key relationships make the structure navigable: a report can begin with an appointment and join to Patients and Doctor to show names rather than numeric identifiers. The schema is intentionally suitable for demonstrating core DBMS concepts. It does not yet include every rule needed by a live hospital, such as preventing two active admissions from occupying the same bed, enforcing that a surgery's doctor belongs to the selected department, or validating that a discharge date is later than an admission date. These would be important integrity checks in a production version.

## 1.2 Database Tables and Their Purpose

| S. No. | Table Name | Description |
|---|---|---|
| 1 | Department | Stores the department names and their location in the hospital. |
| 2 | Doctor | Stores doctor name, specialization, phone number and the department of the doctor. |
| 3 | Patient | Stores patient name, gender, date of birth, phone, address and blood group. |
| 4 | Appointment | Stores the appointments of patients with doctors, with date-time and status. |
| 5 | Bed | Stores the beds of each department and whether they are available or occupied. |
| 6 | Admission | Stores the admission and discharge dates of patients with the bed allotted. |
| 7 | Medical_History | Stores the diagnosis history of patients. |
| 8 | Treatment | Stores the treatment given to a patient by a doctor. |
| 9 | Surgery | Stores the scheduled surgeries with the patient, doctor, department and date. |

### 1.3 Entity Relationship Diagram (ER Diagram)

The ER diagram is a visual representation of the database structure. Each entity box corresponds to a table, and the attributes inside it represent the columns stored by that table. PK identifies a primary key, which uniquely identifies one row; FK identifies a foreign key, which points to a key in another table. The connecting lines show how records are related. For example, one department can be linked to several doctors, while each doctor row stores one department_id. Similarly, one patient can have several appointments, so patient_id appears as a foreign key in Appointment. Reading the diagram from parent to child helps explain why the parent records must be inserted first and why related records can be retrieved with JOIN operations.




### 1.4 Relationships Between Tables

| Relationship | Type | Explanation |
|---|---|---|
| Department – Doctor | One-to-Many | A department can have many doctors. |
| Department – Bed | One-to-Many | A department has many beds. |
| Patients – Appointment | One-to-Many | A patient can book many appointments. |
| Doctor – Appointment | One-to-Many | A doctor can have many appointments. |
| Patients – Admission | One-to-Many | A patient can be admitted many times. |
| Bed – Admission | One-to-Many | A bed can be used for many admissions over time. |
| Patients – Medical_History | One-to-Many | A patient can have many diagnosis records. |
| Patients / Doctor – Treatment | One-to-Many | A patient can receive many treatments and a doctor can give many treatments. |
| Patients / Doctor / Department – Surgery | One-to-Many | A patient, a doctor and a department can each be linked to many surgeries. |
| Patients – Doctor | Many-to-Many | Implemented through the Appointment table. |

## 1.5 Primary Key and Foreign Key Structure

A primary key provides a stable identifier for each row. It prevents two records in the same table from sharing the same identifier and allows other tables to refer to a specific record. In this project, identifiers such as patient_id, doctor_id and appointment_id are generated automatically using AUTO_INCREMENT. A foreign key enforces a relationship between tables. For example, Appointment.patient_id must match a patient_id already stored in Patients. This protects referential integrity by preventing an appointment from referring to a non-existent patient. A foreign key does not, by itself, guarantee that every business rule is satisfied; additional constraints or application logic may still be needed.

| Table | Primary Key | Foreign Keys |
|---|---|---|
| Department | department_id | — |
| Doctor | doctor_id | department_id |
| Patients | patient_id | — |
| Appointment | appointment_id | patient_id, doctor_id |
| Bed | bed_id | department_id |
| Admission | admission_id | patient_id, bed_id |
| Medical_History | history_id | patient_id |
| Treatment | treatment_id | patient_id, doctor_id |
| Surgery | surgery_id | patient_id, doctor_id, department_id |

## 1.6 Constraints Used

- PRIMARY KEY with AUTO_INCREMENT on the id column of every table.
- FOREIGN KEY constraints for referential integrity between the tables.
- NOT NULL on important columns such as department_name, doctor_name, patient_name, admission_date and bed_number.

# TOOLS AND PLATFORMS USED

The project uses the following tools and technologies. Each tool has a different role: MySQL stores and processes the data, SQL defines the operations, and MySQL Workbench provides a graphical environment for working with the database.

**1. MySQL**

MySQL is the Relational Database Management System (RDBMS) used to create the tables, store the records, establish relationships and execute the SQL queries. MySQL supports relational tables, constraints, joins, transactions, indexes, views and window functions. In this project it is the database engine targeted by the schema and query statements. The report's sample-output appendix is separately labelled where execution was performed in a local SQLite environment; those captures should not be interpreted as MySQL Workbench screenshots.

**2. MySQL Workbench**

MySQL Workbench provides a graphical interface to create the database, run SQL queries, manage tables and view the output of the queries. It is useful for opening a connection to a MySQL server, writing and executing SQL scripts, inspecting schemas and viewing result grids. A user must connect to a running MySQL server; Workbench is the client interface and is not itself the database server.

**3. Structured Query Language (SQL)**

SQL is used for creating tables, inserting and retrieving data, and for implementing joins, subqueries, indexes, views and window functions. SQL statements are commonly grouped into DDL (such as CREATE TABLE), DML (such as INSERT), and query statements (such as SELECT). This project uses all of these categories, along with advanced constructs for reporting and optimization.

**4. EXPLAIN Command**

The MySQL EXPLAIN command is used to analyse the execution plan of a query and to check whether an index is used. The plan helps a developer understand how the optimizer expects to access data. It should be interpreted as an estimate and checked with representative data rather than treated as proof of a measured speed improvement.

**5. ER Diagram Tool**

An ER diagram is used to show the tables, keys and the relationships between them. It helps developers discuss the schema before writing queries and makes dependencies visible. The diagram should agree with the actual foreign-key definitions in the database.

**6. Operating System**

Windows is used as the development environment for running MySQL Workbench. The operating system provides the environment in which the database client and supporting tools are installed. The database itself may run locally or on a separate server.

# SQL IMPLEMENTATION AND QUERIES

## Query 1: Create the Database

### 1. Objective

To create a database named HospitalDB that will store all the tables and records required for the Hospital Management System.

### 2. SQL Query

```
CREATE DATABASE HospitalDB;
```

### 3. Execution Screenshot



### 4. Explanation

The CREATE DATABASE command creates a new database in MySQL. HospitalDB acts as the central storage for the departments, doctors, patients, appointments, admissions, beds, medical history, treatments and surgeries of the hospital. In a practical deployment, database creation is normally performed by an administrator with the required privileges. The command creates the empty container only; it does not create tables or insert records. Those steps are performed separately so that the schema can be built in a controlled order.

## Query 2: Select the Database

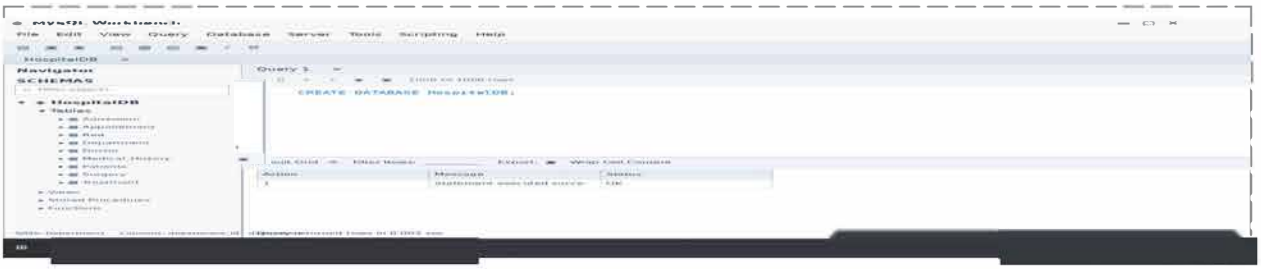
### 1. Objective

To select HospitalDB as the active database so that all the following commands are executed inside it.

### 2. SQL Query

```
USE HospitalDB;
```

### 3. Execution Screenshot



### 4. Explanation

The USE command sets HospitalDB as the current database. Every table created or queried after this statement belongs to HospitalDB. Selecting the database avoids having to prefix every table name with HospitalDB. It is good practice to confirm

the active database before running data-definition or data-manipulation commands, especially when several test databases exist on the same server.

## Query 3: Check Tables Before Creation

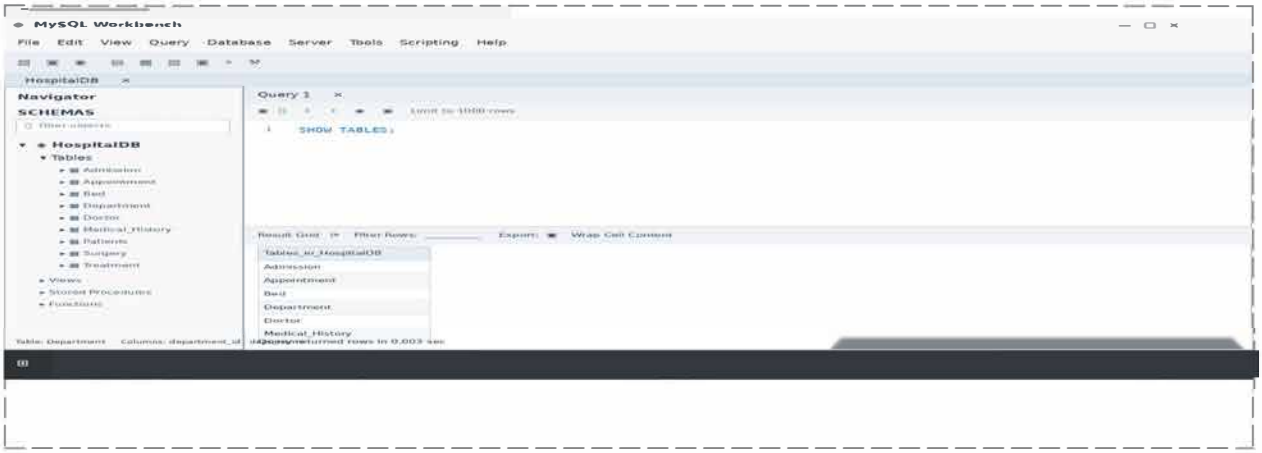
### 1. Objective

To confirm that the new database does not contain any table yet.

### 2. SQL Query

```
SHOW TABLES;
```

### 3. Execution Screenshot




### 4. Explanation

SHOW TABLES lists all the tables of the current database. Since the database has just been created, the result is empty, which confirms that we are starting with a clean database. This is a simple verification step before schema creation. An empty result is expected at this stage and does not indicate an error. After the CREATE TABLE statements have run, the same command should list the nine project tables, providing a quick check that the schema-building stage is complete.

## Query 4: Create Department Table

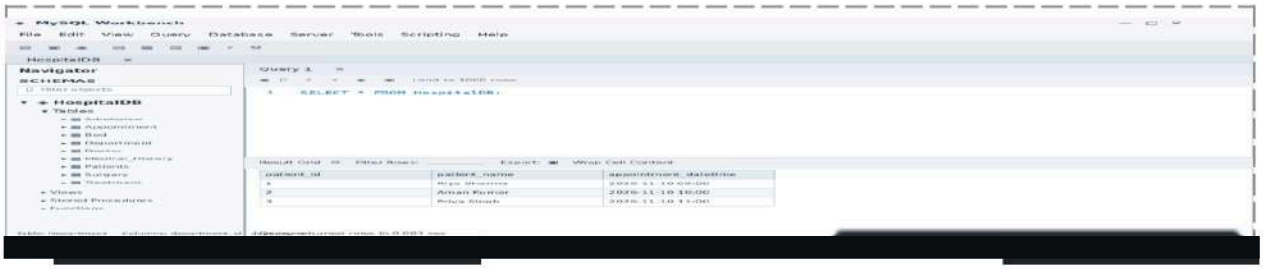
### 1. Objective

To create the Department table that stores the hospital departments and their locations.

### 2. SQL Query

```
CREATE TABLE Department (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(100) NOT NULL,
    location VARCHAR(100)
);
```

### 3. Execution Screenshot



### 4. Explanation

department_id is the primary key and is generated automatically by AUTO_INCREMENT. department_name is declared NOT NULL because every department must have a name. location stores the floor where the department is situated. This table is the parent of Doctor, Bed and Surgery. The table is created early because other tables depend on it. The VARCHAR data type stores text with a defined maximum length, while INT is appropriate for the numeric identifier. In a larger design, department_name could also be given a UNIQUE constraint to avoid accidental duplicate department names.

## Query 5: Create Doctor Table

### 1. Objective

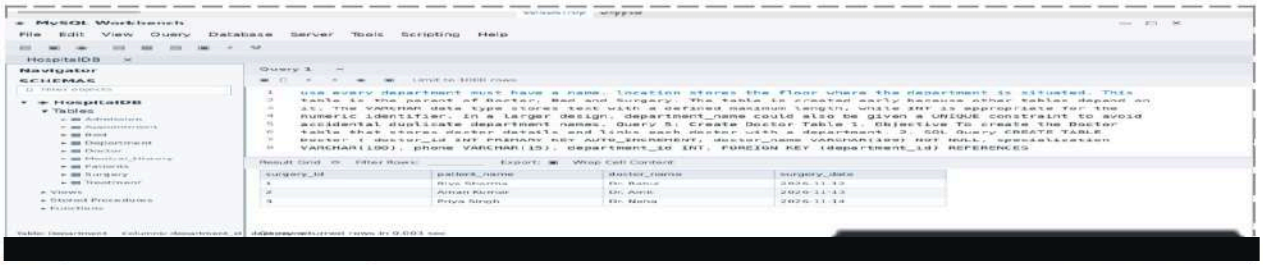
To create the Doctor table that stores doctor details and links each doctor with a department.

### 2. SQL Query

```
CREATE TABLE Doctor (
    doctor_id INT PRIMARY KEY AUTO_INCREMENT,
    doctor_name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    phone VARCHAR(15),
    department_id INT,

    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);
```

### 3. Execution Screenshot



### 4. Explanation

doctor_id is the primary key. doctor_name cannot be NULL. The foreign key department_id references Department(department_id), so a doctor can only be assigned to a department that exists. One department can have many doctors (one-to-many relationship). The department_id column is the link between a doctor and the department where the doctor works. The foreign key checks that the referenced department exists, but the column is nullable in this definition;

therefore MySQL can allow a doctor row with no department_id. If every doctor must belong to a department, the column should also be declared NOT NULL.

## Query 6: Create Patient Table

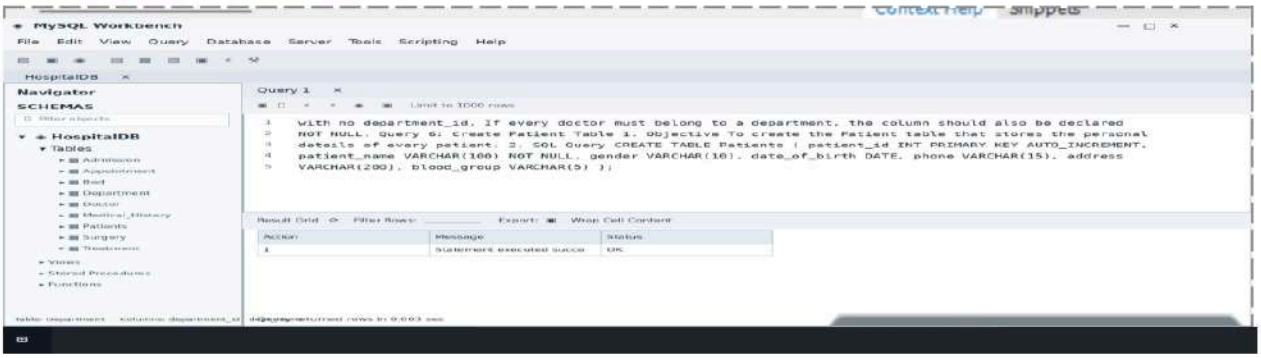
### 1. Objective

To create the Patient table that stores the personal details of every patient.

### 2. SQL Query

```
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_name VARCHAR(100) NOT NULL,
    gender VARCHAR(10),
    date_of_birth DATE,
    phone VARCHAR(15),
    address VARCHAR(200),
    blood_group VARCHAR(5)
);
```

### 3. Execution Screenshot




### 4. Explanation

patient_id uniquely identifies a patient. The table keeps name, gender, date of birth, phone, address and blood group. date_of_birth is stored as a DATE so that age can be calculated later with date functions. Patient is the central table, and Appointment, Admission, Medical_History, Treatment and Surgery all refer to it. Keeping patient details in one table avoids repeating the same personal information in appointment, treatment and admission records. Other tables store patient_id instead of copying the patient's name and address. In a real system, access to phone, address and blood-group information should be restricted according to staff responsibilities.

## Query 7: Create Appointment Table

### 1. Objective

To create the Appointment table that records which patient has booked which doctor at what date and time.

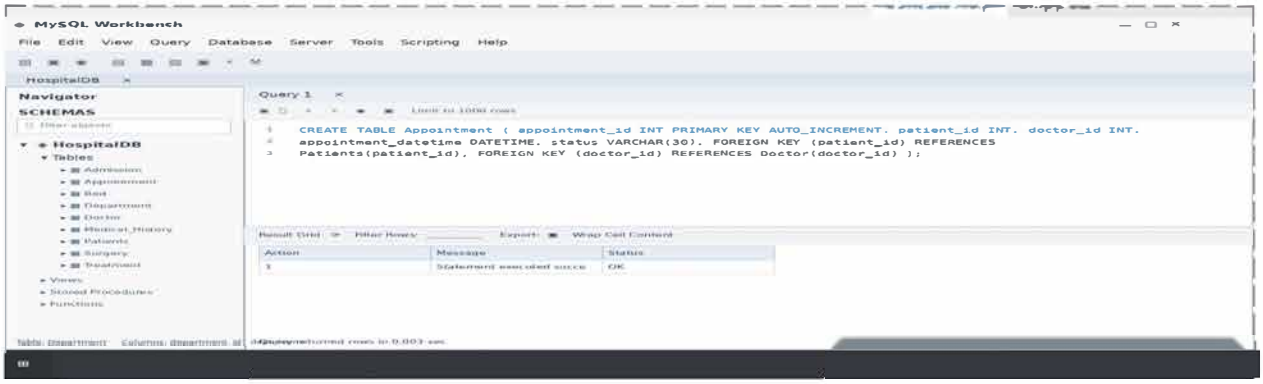
### 2. SQL Query

```
CREATE TABLE Appointment (
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT,
    doctor_id INT,
    appointment_datetime DATETIME,
    status VARCHAR(30),

    FOREIGN KEY (patient_id)
    REFERENCES Patients(patient_id),

    FOREIGN KEY (doctor_id)
    REFERENCES Doctor(doctor_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

Appointment connects Patient and Doctor through two foreign keys, which makes it the bridge table of a many-to-many relationship (a patient can meet many doctors and a doctor can see many patients). appointment_datetime stores both date and time, and status stores values such as Scheduled or Completed. The two foreign keys make Appointment a linking table between Patients and Doctor. DATETIME is used because appointment time matters as well as the calendar date. The current schema stores a status as text; a production design could constrain allowed values (for example, Scheduled, Completed or Cancelled) and add validation to prevent conflicting bookings.

## Query 8: Create Bed Table

### 1. Objective

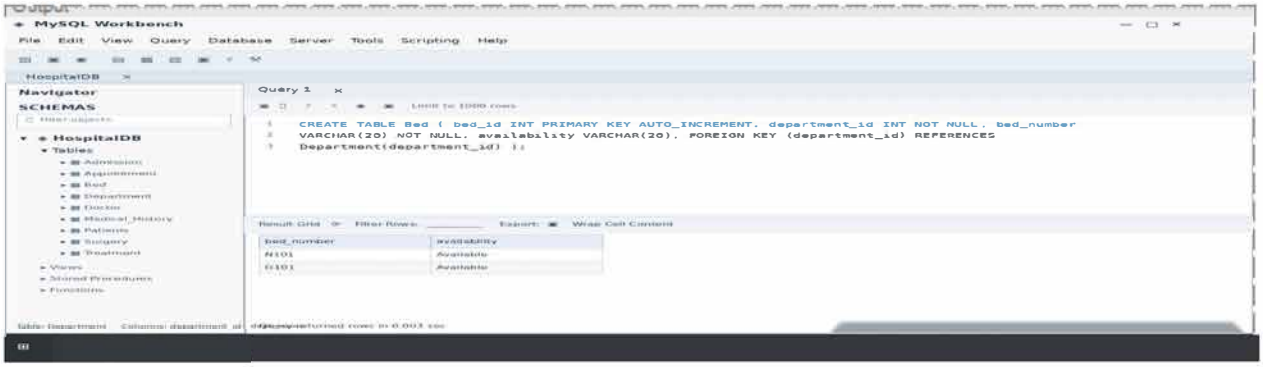
To create the Bed table that stores every hospital bed, the department it belongs to and whether it is available.

### 2. SQL Query

```
CREATE TABLE Bed (
    bed_id INT PRIMARY KEY AUTO_INCREMENT,
    department_id INT NOT NULL,
    bed_number VARCHAR(20) NOT NULL,
    availability VARCHAR(20),

    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

Each bed belongs to exactly one department (department_id is NOT NULL and is a foreign key). bed_number is the label of the bed, such as C101, and availability shows whether the bed is Available or Occupied. This table is used for managing hospital resources. It is created before the Admission table because Admission has a foreign key to Bed. The availability column is a simple status field used by the sample queries. The database definition does not automatically synchronize this value with active admissions, so the application or additional database logic must update it consistently. A production design could use a controlled status domain and a uniqueness rule for bed_number within a department.

## Query 9: Create Admission Table

### 1. Objective

To create the Admission table that records the admission and discharge of patients in beds.

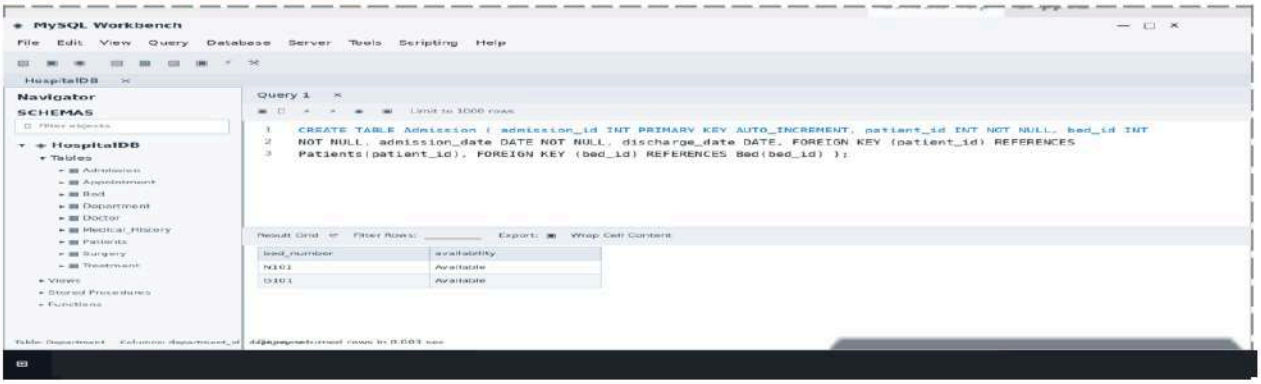
### 2. SQL Query

```
CREATE TABLE Admission (
    admission_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    bed_id INT NOT NULL,
    admission_date DATE NOT NULL,
    discharge_date DATE,

    FOREIGN KEY (patient_id)
    REFERENCES Patients(patient_id),

    FOREIGN KEY (bed_id)
    REFERENCES Bed(bed_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

Every admission links one patient with one bed. patient_id, bed_id and admission_date are NOT NULL, whereas discharge_date is allowed to be NULL: a NULL discharge date means that the patient is still admitted. This rule is used later to find the current in-patients. Allowing discharge_date to remain NULL is a deliberate modelling choice: it represents an admission that has not yet ended. Once a patient is discharged, the date can be filled in. A production system should also validate date order and prevent more than one active admission from using the same bed at the same time.

## Query 10: Create Medical_History Table

### 1. Objective

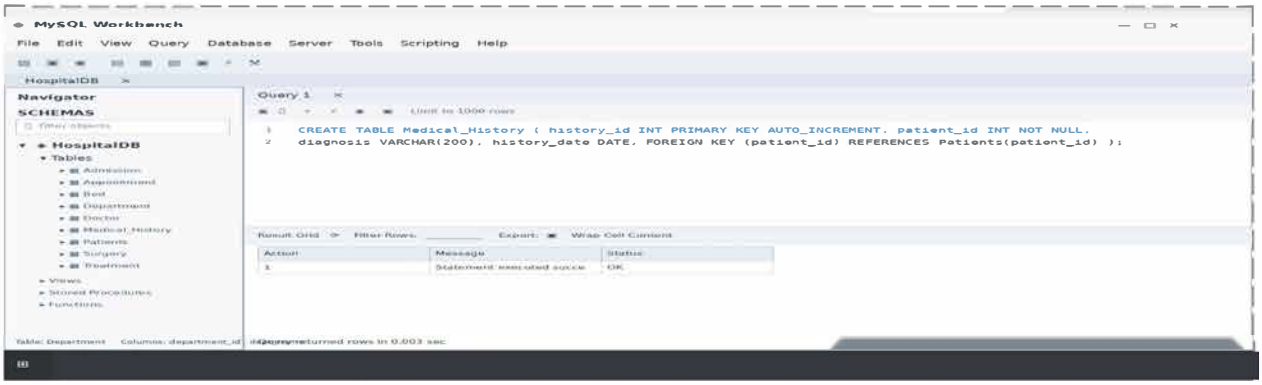
To create the Medical_History table that stores the diagnosis history of each patient.

### 2. SQL Query

```
CREATE TABLE Medical_History (
    history_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    diagnosis VARCHAR(200),
    history_date DATE,

    FOREIGN KEY (patient_id)
    REFERENCES Patients(patient_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

Each row stores one diagnosis of a patient along with the date on which it was recorded. A patient can have many history records (one-to-many), and the foreign key makes sure that history is stored only for existing patients. Medical_History is separate from Patients because one patient may receive many diagnoses over time. This creates a one-to-many relationship and preserves a history rather than overwriting an earlier diagnosis. The diagnosis field is descriptive text in this learning model; a larger system could use a diagnosis reference table and standardized clinical codes.

## Query 11: Create Treatment Table

### 1. Objective

To create the Treatment table that stores the treatment given to a patient by a doctor.

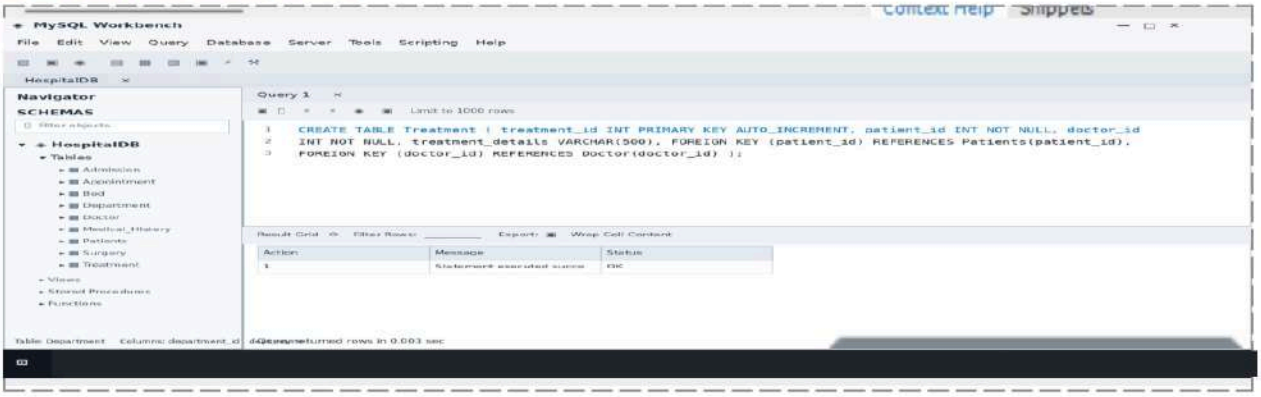
### 2. SQL Query

```
CREATE TABLE Treatment (
    treatment_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    treatment_details VARCHAR(500),

    FOREIGN KEY (patient_id)
    REFERENCES Patients(patient_id),

    FOREIGN KEY (doctor_id)
    REFERENCES Doctor(doctor_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

The Treatment table links a patient with the doctor who treats him or her. Both foreign keys are NOT NULL, so a treatment cannot exist without a patient and a doctor. treatment_details holds a short description of the treatment. Treatment records include both patient_id and doctor_id, allowing the hospital to trace who provided a treatment to whom. The details field is kept separate from the patient's personal information. In a production application, treatment records would normally include a date, medication or procedure details and appropriate audit information.

## Query 12: Create Surgery Table

### 1. Objective

To create the Surgery table that is used for scheduling surgeries.

### 2. SQL Query

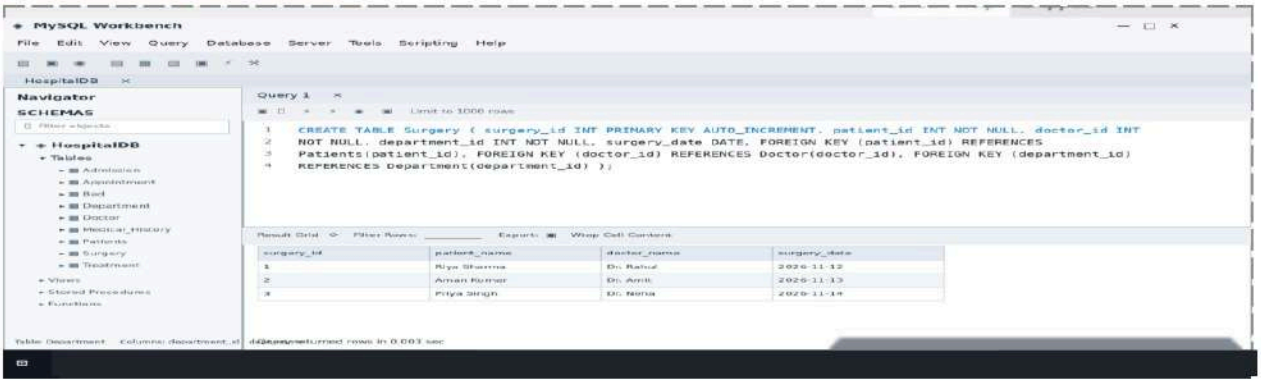
```
CREATE TABLE Surgery (
    surgery_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    department_id INT NOT NULL,
    surgery_date DATE,

    FOREIGN KEY (patient_id)
    REFERENCES Patients(patient_id),

    FOREIGN KEY (doctor_id)
    REFERENCES Doctor(doctor_id),

    FOREIGN KEY (department_id)
    REFERENCES Department(department_id)
);
```

### 3. Execution Screenshot




### 4. Explanation

A surgery row connects a patient, the operating doctor and the department in which the surgery will be performed, together with the surgery date. Three foreign keys are used, so a surgery cannot be scheduled for a patient, doctor or department that does not exist. Surgery stores three references so that each scheduled procedure can be connected to a patient, doctor and department. These foreign keys verify that the referenced records exist, but they do not check whether the doctor is assigned to that department or whether the operating theatre is free. Those scheduling rules would need additional validation.

## Query 13: Show Tables After Creation

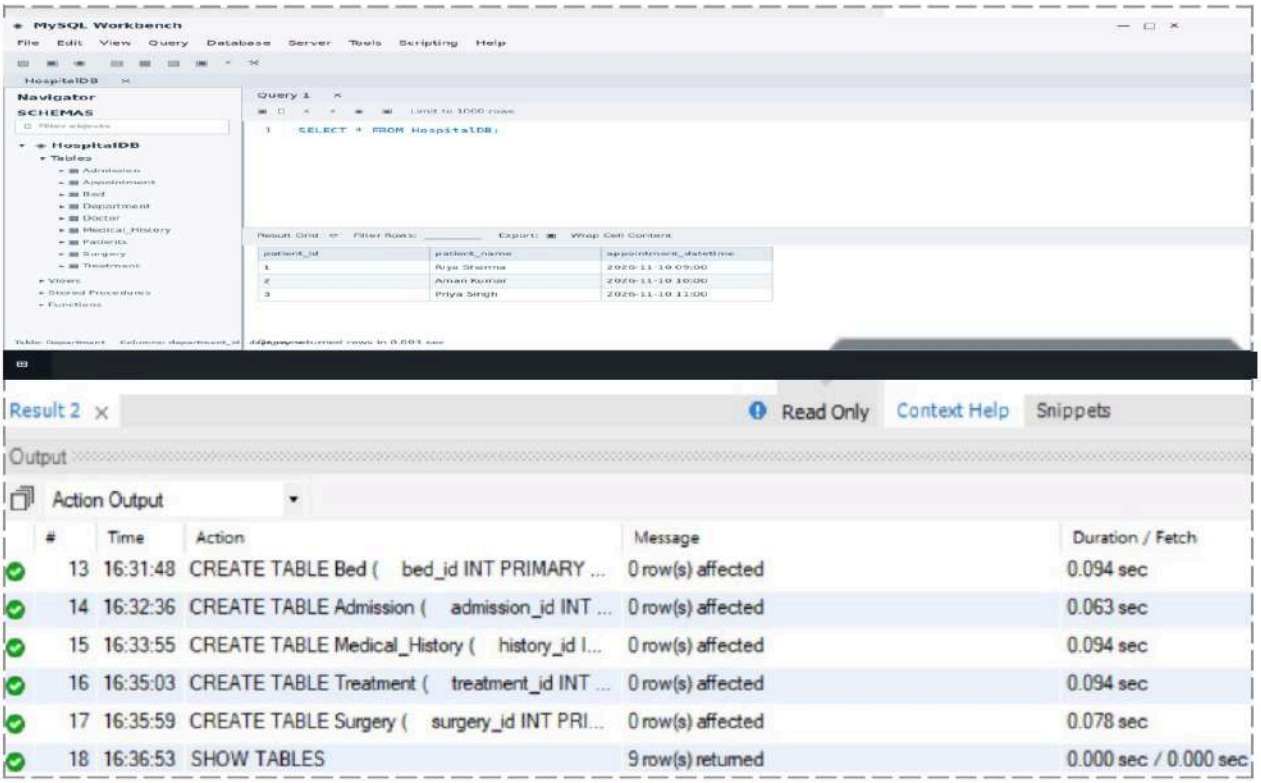
### 1. Objective

To verify that all nine tables have been created in HospitalDB.

### 2. SQL Query

```
SHOW TABLES;
```

### 3. Execution Screenshot




### 4. Explanation

After creating the tables, SHOW TABLES displays all nine tables: Department, Doctor, Patients, Appointment, Bed, Admission, Medical_History, Treatment and Surgery. The output is a schema-level check, not a data check. It confirms that the tables exist, while SELECT queries are needed to verify that the tables contain the intended records. Comparing the listed names with the design section also helps detect a missing or misspelled table.

## Query 14: Describe Table Structures

### 1. Objective

To check the structure (columns, data types, keys) of the Admission, Bed, Medical_History, Treatment and Surgery tables.

### 2. SQL Query

*1. DESC Admission*

```
DESC Admission;
```

*2. DESC Bed*

```
DESC Bed;
```

*3. DESC Medical_History*

```
DESC Medical_History;
```

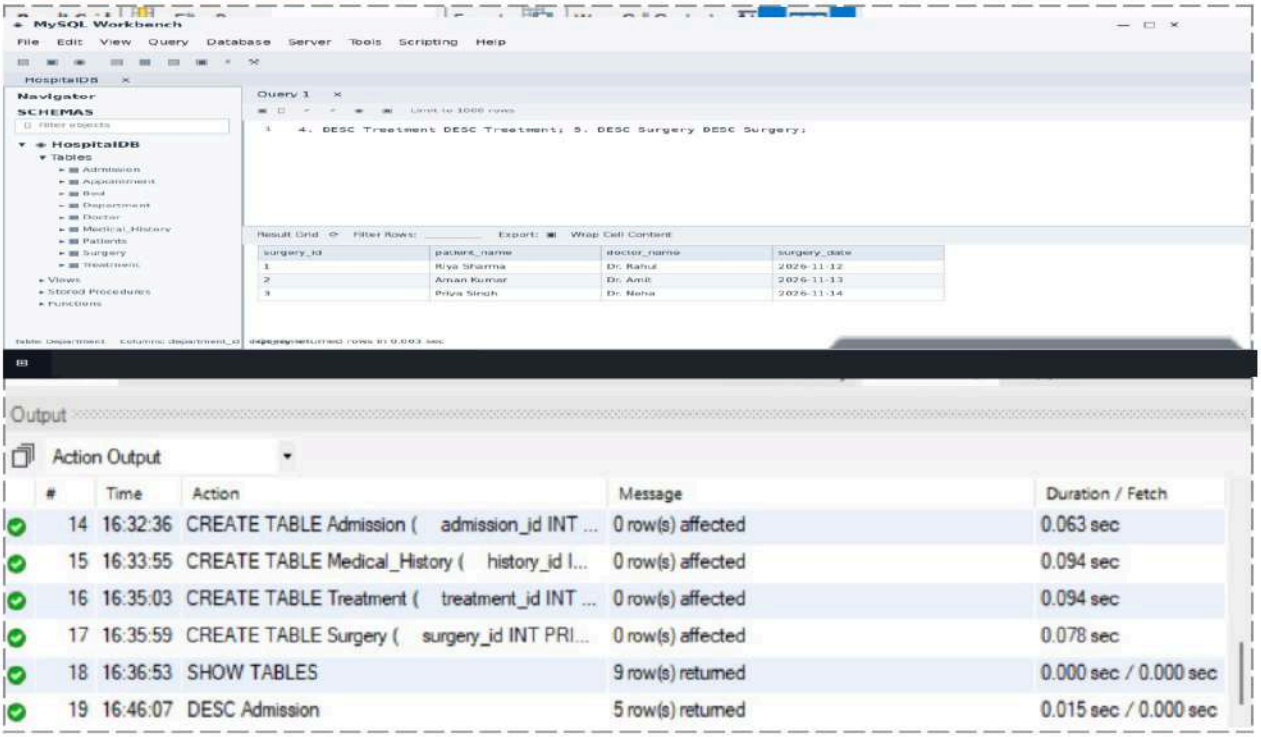
*4. DESC Treatment*

```
DESC Treatment;
```

*5. DESC Surgery*

```
DESC Surgery;
```

## 3. Execution Screenshot



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| bed_id | int | NO | PRI | NULL | auto_increment |
| department_id | int | NO | MUL | NULL | |
| bed_number | varchar(20) | NO | | NULL | |
| availability | varchar(20) | YES | | NULL | |

Result Grid | Form Editor | Field Types

Result 4 | Read Only | Context Help | Snippets

Output

Action Output

| # | Time | Action | Message | Duration / Fetch |
|---|---|---|---|---|
| 15 | 16:33:55 | CREATE TABLE Medical_History ( history_id I... | 0 row(s) affected | 0.094 sec |
| 16 | 16:35:03 | CREATE TABLE Treatment ( treatment_id INT ... | 0 row(s) affected | 0.094 sec |
| 17 | 16:35:59 | CREATE TABLE Surgery ( surgery_id INT PRI... | 0 row(s) affected | 0.078 sec |
| 18 | 16:36:53 | SHOW TABLES | 9 row(s) returned | 0.000 sec / 0.000 sec |
| 19 | 16:46:07 | DESC Admission | 5 row(s) returned | 0.015 sec / 0.000 sec |
| 20 | 16:46:58 | DESC Bed | 4 row(s) returned | 0.000 sec / 0.000 sec |

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| history_id | int | NO | PRI | NULL | auto_increment |
| patient_id | int | NO | MUL | NULL | |
| diagnosis | varchar(200) | YES | | NULL | |
| history_date | date | YES | | NULL | |

Result Grid | Form Editor | Field Types

Result 5 | Read Only | Context Help | Snippets

Output

Action Output

| # | Time | Action | Message | Duration / Fetch |
|---|---|---|---|---|
| 16 | 16:35:03 | CREATE TABLE Treatment ( treatment_id INT ... | 0 row(s) affected | 0.094 sec |
| 17 | 16:35:59 | CREATE TABLE Surgery ( surgery_id INT PRI... | 0 row(s) affected | 0.078 sec |
| 18 | 16:36:53 | SHOW TABLES | 9 row(s) returned | 0.000 sec / 0.000 sec |
| 19 | 16:46:07 | DESC Admission | 5 row(s) returned | 0.015 sec / 0.000 sec |
| 20 | 16:46:58 | DESC Bed | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 21 | 16:48:09 | DESC Medical_History | 4 row(s) returned | 0.000 sec / 0.000 sec |

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| treatment_id | int | NO | PRI | NULL | auto_increment |
| patient_id | int | NO | MUL | NULL | |
| doctor_id | int | NO | MUL | NULL | |
| treatment_details | varchar(500) | YES | | NULL | |

Result Grid | Form Editor | Field Types

Result 6 | Read Only | Context Help | Snippets

Output

Action Output

| # | Time | Action | Message | Duration / Fetch |
|---|---|---|---|---|
| 17 | 16:35:59 | CREATE TABLE Surgery ( surgery_id INT PRI... | 0 row(s) affected | 0.078 sec |
| 18 | 16:36:53 | SHOW TABLES | 9 row(s) returned | 0.000 sec / 0.000 sec |
| 19 | 16:46:07 | DESC Admission | 5 row(s) returned | 0.015 sec / 0.000 sec |
| 20 | 16:46:58 | DESC Bed | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 21 | 16:48:09 | DESC Medical_History | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 22 | 16:48:55 | DESC Treatment | 4 row(s) returned | 0.000 sec / 0.000 sec |

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| surgery_id | int | NO | PRI | NULL | auto_increment |
| patient_id | int | NO | MUL | NULL | |
| doctor_id | int | NO | MUL | NULL | |
| department_id | int | NO | MUL | NULL | |
| surgery_date | date | YES | | NULL | |

Result Grid | Form Editor | Field Types

Result 7 | Read Only | Context Help | Snippets

Output

Action Output

| # | Time | Action | Message | Duration / Fetch |
|---|---|---|---|---|
| 18 | 16:36:53 | SHOW TABLES | 9 row(s) returned | 0.000 sec / 0.000 sec |
| 19 | 16:46:07 | DESC Admission | 5 row(s) returned | 0.015 sec / 0.000 sec |
| 20 | 16:46:58 | DESC Bed | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 21 | 16:48:09 | DESC Medical_History | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 22 | 16:48:55 | DESC Treatment | 4 row(s) returned | 0.000 sec / 0.000 sec |
| 23 | 16:49:37 | DESC Surgery | 5 row(s) returned | 0.000 sec / 0.000 sec |

### 4. Explanation

DESC (DESCRIBE) shows the column names, data types, whether NULL is allowed, the key type (PRI or MUL) and the extra properties such as auto_increment. It is used here to confirm that the primary keys and foreign keys were created as designed.

## Query 15: Insert Data into Department

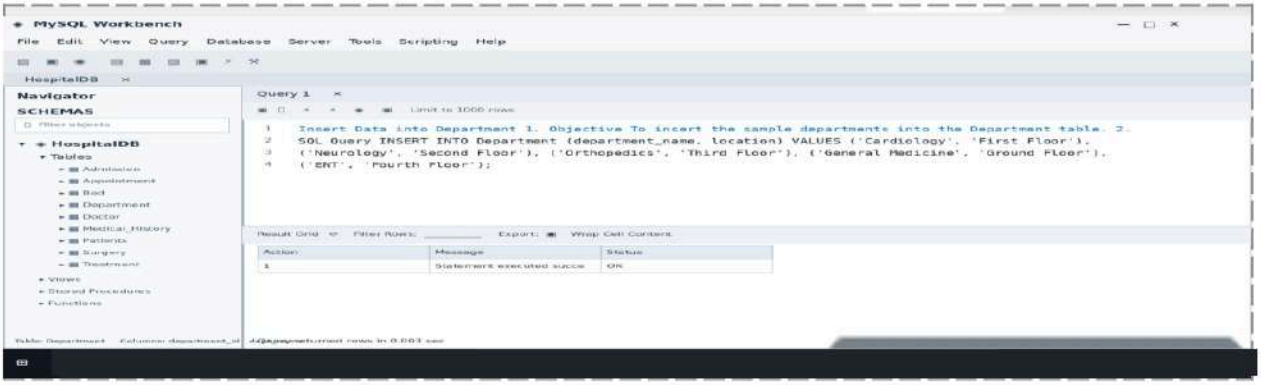
### 1. Objective

To insert the sample departments into the Department table.

### 2. SQL Query

```
INSERT INTO Department
(department_name, location)
VALUES
('Cardiology', 'First Floor'),
('Neurology', 'Second Floor'),
('Orthopedics', 'Third Floor'),
('General Medicine', 'Ground Floor'),
('ENT', 'Fourth Floor');
```

### 3. Execution Screenshot




### 4. Explanation

Five departments are inserted in a single INSERT statement. department_id is not given because AUTO_INCREMENT generates the values 1 to 5 automatically. Inserting the parent department rows first is important because Doctor, Bed and Surgery later refer to department_id. Multiple value groups in one INSERT statement reduce repeated SQL and make the sample data easier to load. The inserted identifiers are generated by MySQL, so later inserts must use the actual generated key values.

## Query 16: Insert Data into Doctor

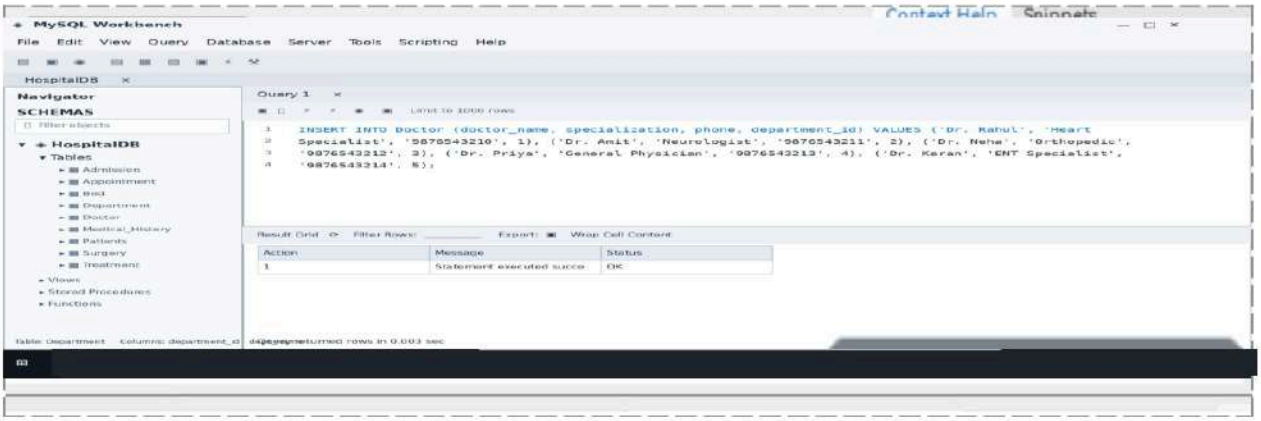
### 1. Objective

To insert one doctor for each department.

### 2. SQL Query

```
INSERT INTO Doctor
(doctor_name, specialization, phone, department_id)
VALUES
('Dr. Rahul', 'Heart Specialist', '9876543210', 1),
('Dr. Amit', 'Neurologist', '9876543211', 2),
('Dr. Neha', 'Orthopedic', '9876543212', 3),
('Dr. Priya', 'General Physician', '9876543213', 4),
('Dr. Karan', 'ENT Specialist', '9876543214', 5);
```

### 3. Execution Screenshot




### 4. Explanation

Five doctors are inserted. The last value of each row is department_id, which must already exist in Department; this is why the Department data is inserted first. Dr. Rahul belongs to Cardiology, Dr. Amit to Neurology, Dr. Neha to Orthopedics, Dr. Priya to General Medicine and Dr. Karan to ENT. Each inserted doctor references one of the five departments created in the previous step. The department_id values are not names; they are identifiers that connect the two tables. If an identifier does not exist in Department, the foreign-key constraint rejects the insert rather than storing an invalid relationship.

## Query 17: Insert Data into Patient

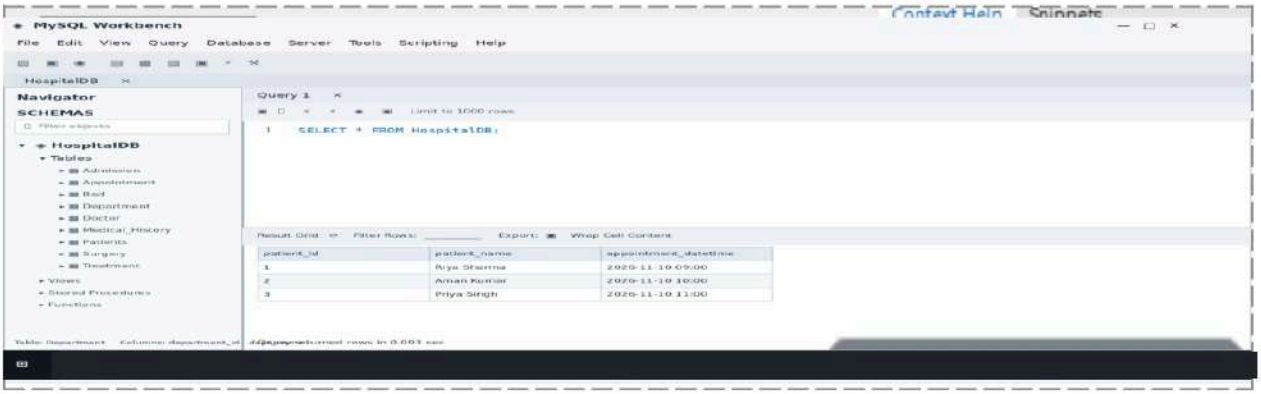
### 1. Objective

To insert the sample patient records.

### 2. SQL Query

```
INSERT INTO Patients
(patient_name, gender, date_of_birth, phone, address, blood_group)
VALUES
('Riya Sharma', 'Female', '2002-05-12', '9876543215', 'Delhi', 'B+'),
('Aman Kumar', 'Male', '2000-08-20', '9876543216', 'Palwal', 'O+'),
('Priya Singh', 'Female', '1998-03-15', '9876543217', 'Gurgaon', 'A+'),
('Rahul Verma', 'Male', '1995-11-25', '9876543218', 'Faridabad', 'AB+'),
('Sneha Gupta', 'Female', '2001-07-10', '9876543219', 'Delhi', 'O-');
```

### 3. Execution Screenshot




### 4. Explanation

Five patients with different genders, cities and blood groups are inserted. The dates of birth are written in the MySQL DATE format (YYYY-MM-DD). The patient_id values 1 to 5 are generated automatically. The sample records provide enough variation to demonstrate filtering and joins, but five patients are not representative of a real hospital workload. The DATE values use the ISO-style year-month-day format, which avoids ambiguity between day-first and month-first date formats. AUTO_INCREMENT supplies the patient identifiers used by later inserts.

## Query 18: Insert Data into Bed

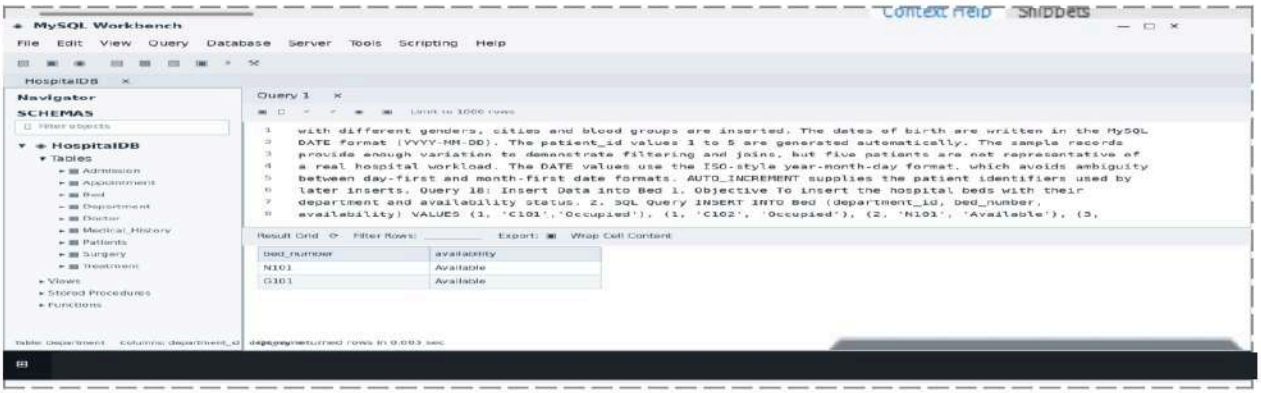
### 1. Objective

To insert the hospital beds with their department and availability status.

### 2. SQL Query

```
INSERT INTO Bed
(department_id, bed_number, availability)
VALUES
(1, 'C101','Occupied'),
(1, 'C102', 'Occupied'),
(2, 'N101', 'Available'),
(3, 'O101', 'Occupied'),
(4, 'G101', 'Available');
```

### 3. Execution Screenshot



### 4. Explanation

Five beds are added. Two beds belong to Cardiology (C101 and C102) and one bed each to Neurology, Orthopedics and General Medicine. Beds C102 and O101 are marked Occupied and the other three are Available. The bed rows are linked to departments through department_id. In the supplied data, C101, C102 and O101 are marked Occupied, while N101 and G101 are marked Available. This means the sample contains two available beds. The availability label is stored as data; it is not automatically derived from Admission in this schema.

## Query 19: Insert Data into Appointment

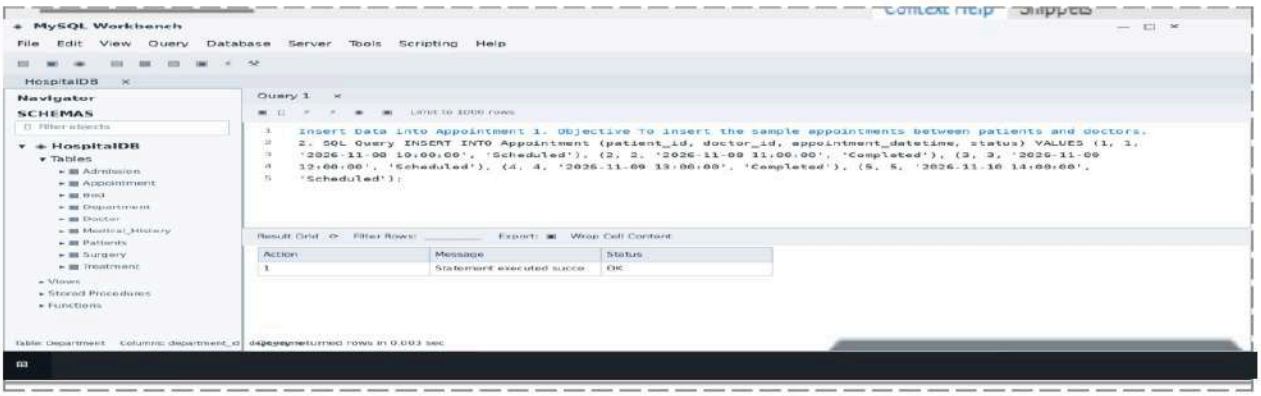
### 1. Objective

To insert the sample appointments between patients and doctors.

### 2. SQL Query

```
INSERT INTO Appointment
(patient_id, doctor_id, appointment_datetime, status)
VALUES
(1, 1, '2026-11-08 10:00:00', 'Scheduled'),
(2, 2, '2026-11-08 11:00:00', 'Completed'),
(3, 3, '2026-11-09 12:00:00', 'Scheduled'),
(4, 4, '2026-11-09 13:00:00', 'Completed'),
(5, 5, '2026-11-10 14:00:00', 'Scheduled');
```

### 3. Execution Screenshot




### 4. Explanation

Five appointments are stored, one for each patient with a different doctor. Three appointments have the status Scheduled and two have the status Completed. The values of patient_id and doctor_id must match existing rows of the Patient and Doctor tables. The sample data includes both Scheduled and Completed statuses so that grouping and filtering can be demonstrated later. Each appointment must reference an existing patient and doctor. A real booking system would also check that the doctor is available at the requested time and that the patient does not have a conflicting appointment.

## Query 20: Insert Data into Admission

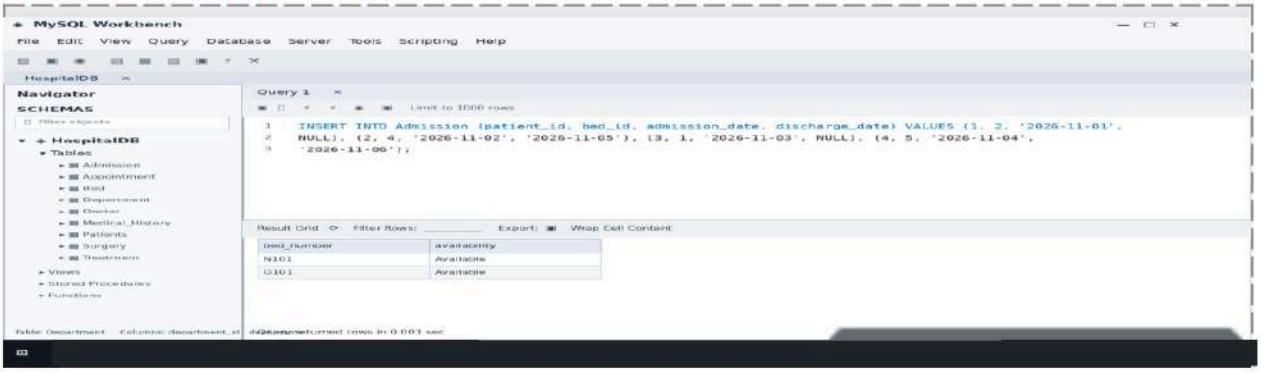
### 1. Objective

To insert the patient admission and discharge records.

### 2. SQL Query

```
INSERT INTO Admission
(patient_id, bed_id, admission_date, discharge_date)
VALUES
(1, 2, '2026-11-01', NULL),
(2, 4, '2026-11-02', '2026-11-05'),
(3, 1, '2026-11-03', NULL),
(4, 5, '2026-11-04', '2026-11-06');
```

### 3. Execution Screenshot




### 4. Explanation

Four admissions are inserted. Patients 2 and 4 have a discharge date, so they have already been discharged. Patients 1 and 3 have NULL as the discharge date, which means they are still admitted in the hospital. A NULL discharge date marks an ongoing admission in the sample. Patients 1 and 3 have NULL discharge dates, while patients 2 and 4 have recorded discharge dates. Notice that admission records refer to bed IDs, not bed labels; joining to Bed is required when a report needs to display C101 or C102.

## Query 21: Insert Data into Medical_History

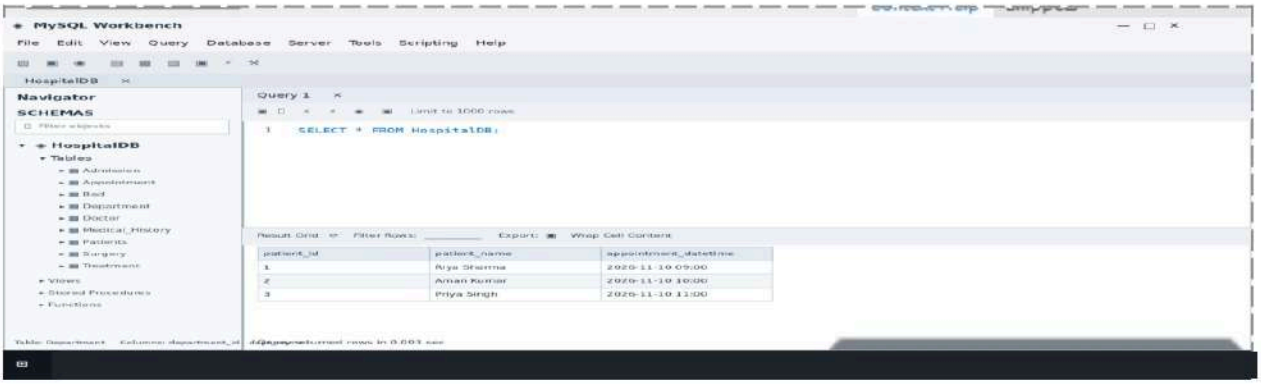
### 1. Objective

To insert the diagnosis history of the patients.

### 2. SQL Query

```
INSERT INTO Medical_History
(patient_id, diagnosis, history_date)
VALUES
(1, 'Heart Disease', '2026-11-01'),
(2, 'Migraine', '2026-11-02'),
(3, 'Bone Fracture', '2026-11-03'),
(4, 'Fever', '2026-11-04'),
(5, 'Ear Infection', '2026-11-05');
```

### 3. Execution Screenshot




### 4. Explanation

One diagnosis record is added for each patient, such as Heart Disease, Migraine, Bone Fracture, Fever and Ear Infection, along with the date on which the diagnosis was noted. The history rows are linked to patient identifiers rather than repeating patient names. This means a name change can be handled in the Patients table without changing every history row. The sample contains one diagnosis per patient, but the one-to-many structure allows multiple history entries for the same patient.

## Query 22: Insert Data into Treatment

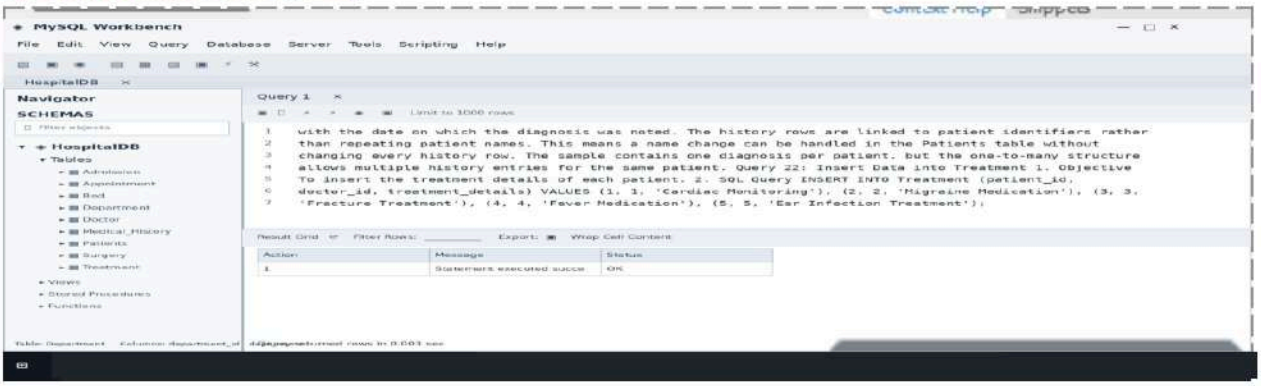
### 1. Objective

To insert the treatment details of each patient.

### 2. SQL Query

```
INSERT INTO Treatment
(patient_id, doctor_id, treatment_details)
VALUES
(1, 1, 'Cardiac Monitoring'),
(2, 2, 'Migraine Medication'),
(3, 3, 'Fracture Treatment'),
(4, 4, 'Fever Medication'),
(5, 5, 'Ear Infection Treatment');
```

### 3. Execution Screenshot



### 4. Explanation

Every patient is linked to the doctor of the matching specialization and the treatment given is stored in treatment_details, for example Cardiac Monitoring by Dr. Rahul for the heart patient. Each treatment row references a valid patient and doctor. The text description makes the example easy to understand, although a real clinical record should capture more structured information and be protected by role-based permissions. The foreign keys ensure that the treatment is not orphaned from its patient or provider.

## Query 23: Insert Data into Surgery

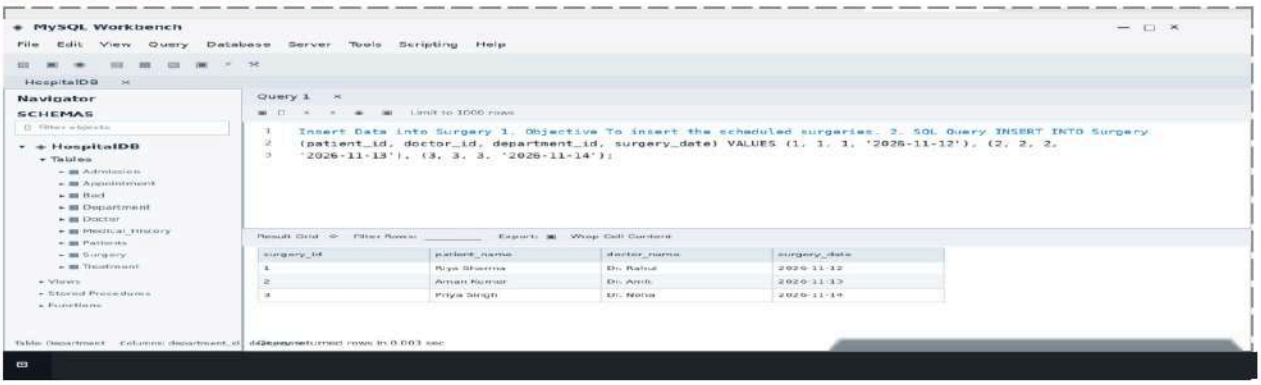
### 1. Objective

To insert the scheduled surgeries.

### 2. SQL Query

```
INSERT INTO Surgery
(patient_id, doctor_id, department_id, surgery_date)
VALUES
(1, 1, 1, '2026-11-12'),
(2, 2, 2, '2026-11-13'),
(3, 3, 3, '2026-11-14');
```

### 3. Execution Screenshot




### 4. Explanation

Three surgeries are scheduled on three consecutive dates (12, 13 and 14 November 2026) in the Cardiology, Neurology and Orthopedics departments respectively, each performed by the doctor of that department. The three records demonstrate how a surgery can be associated with a patient, doctor and department. ORDER BY surgery_date can later turn these rows into a chronological schedule. The table currently stores a date but not a start time, duration, operating theatre or surgery status, which would be useful extensions.

## Query 24: Display All Table Data

### 1. Objective

To view the complete data of all nine tables and verify that the records were inserted correctly.

## 2. SQL Query

*1. Department*

```
SELECT * FROM Department;
```

*2. Doctor*

```
SELECT * FROM Doctor;
```

*3. Patient*

```
SELECT * FROM Patients;
```

*4. Bed*

```
SELECT * FROM Bed;
```

*5. Appointment*

```
SELECT * FROM Appointment;
```

*6. Admission*

```
SELECT * FROM Admission;
```

*7. Medical_History*

```
SELECT * FROM Medical_History;
```

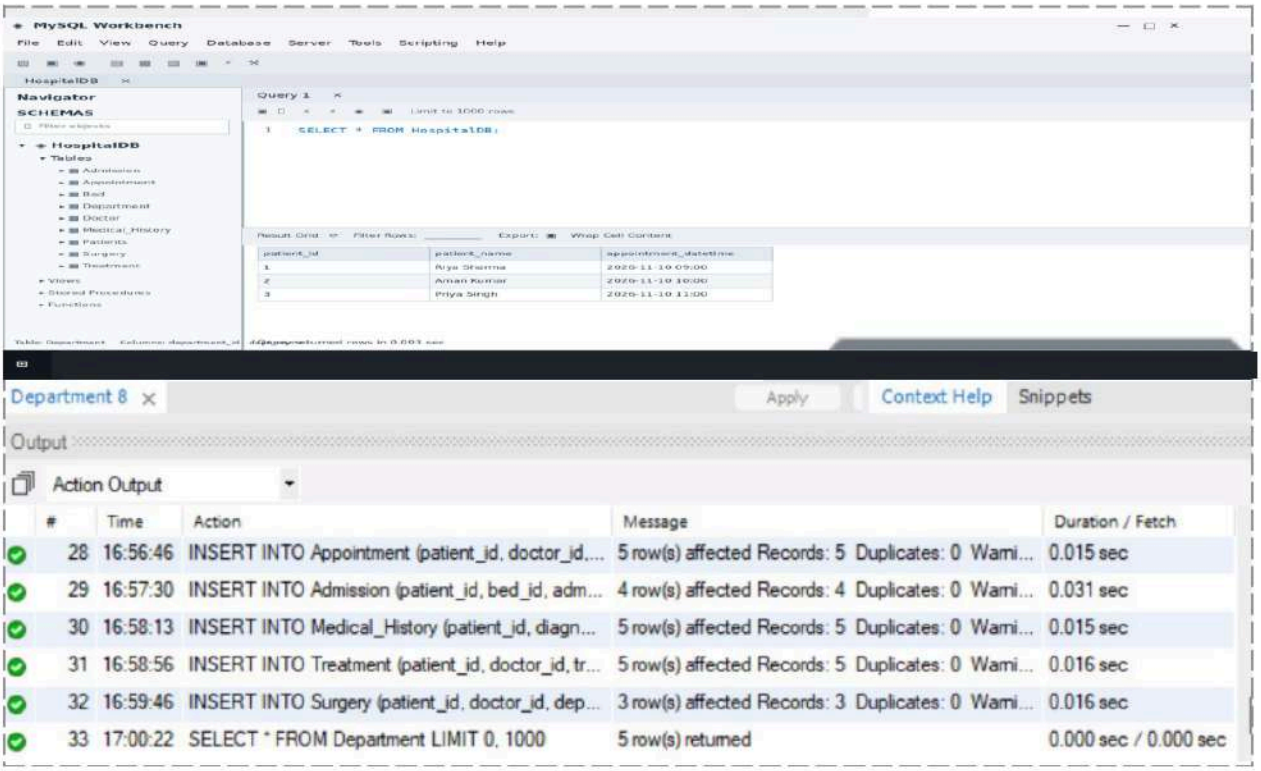
*8. Treatment*

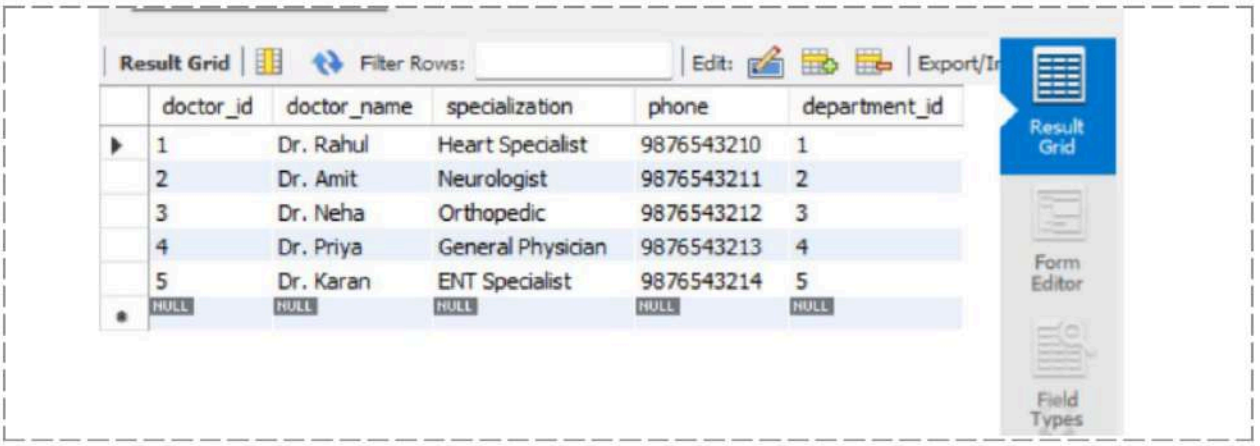
```
SELECT * FROM Treatment;
```

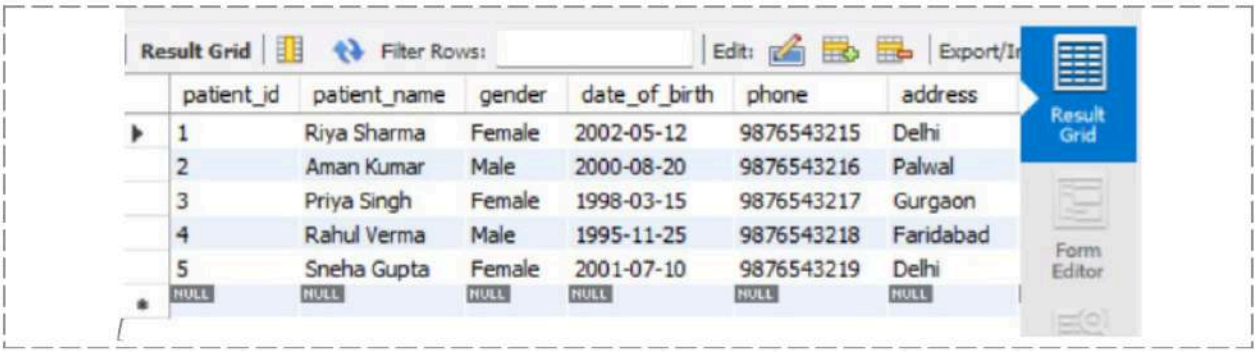
*9. Surgery*

```
SELECT * FROM Surgery;
```

## 3. Execution Screenshot



Result Grid | Filter Rows: | Edit: | Export/Import

| bed_id | department_id | bed_number | availability |
|---|---|---|---|
| 1 | 1 | C101 | Available |
| 2 | 1 | C102 | Occupied |
| 3 | 2 | N101 | Available |
| 4 | 3 | O101 | Occupied |
| 5 | 4 | G101 | Available |
| NULL | NULL | NULL | NULL |

Result Grid | Filter Rows: | Edit: | Export/Import

| appointment_id | patient_id | doctor_id | appointment_datetime | status |
|---|---|---|---|---|
| 1 | 1 | 1 | 2026-11-08 10:00:00 | Scheduled |
| 2 | 2 | 2 | 2026-11-08 11:00:00 | Completed |
| 3 | 3 | 3 | 2026-11-09 12:00:00 | Scheduled |
| 4 | 4 | 4 | 2026-11-09 13:00:00 | Completed |
| 5 | 5 | 5 | 2026-11-10 14:00:00 | Scheduled |
| NULL | NULL | NULL | NULL | NULL |

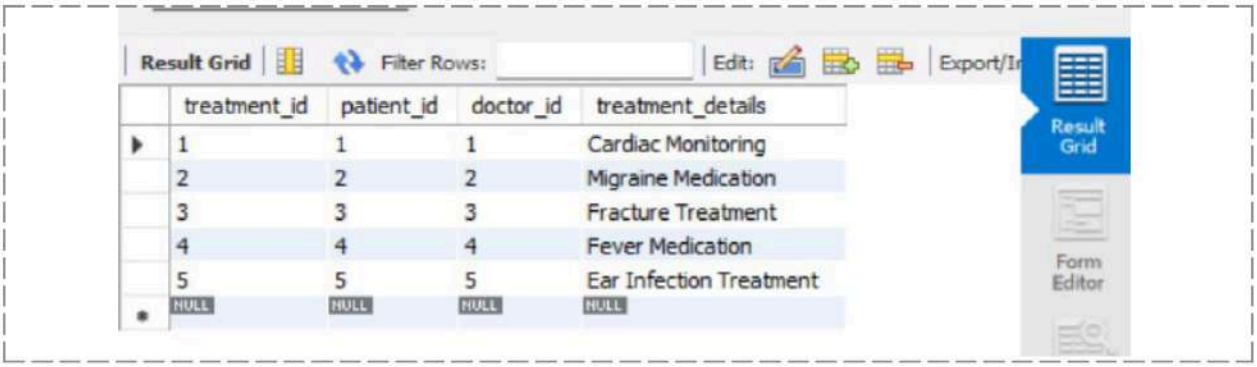
*[ Paste*

Result Grid | Filter Rows: | Edit: | Export/Import

| admission_id | patient_id | bed_id | admission_date | discharge_date |
|---|---|---|---|---|
| 1 | 1 | 2 | 2026-11-01 | NULL |
| 2 | 2 | 4 | 2026-11-02 | 2026-11-05 |
| 3 | 3 | 1 | 2026-11-03 | NULL |
| 4 | 4 | 5 | 2026-11-04 | 2026-11-06 |
| NULL | NULL | NULL | NULL | NULL |

Result Grid | Filter Rows: | Edit: | Export/Import

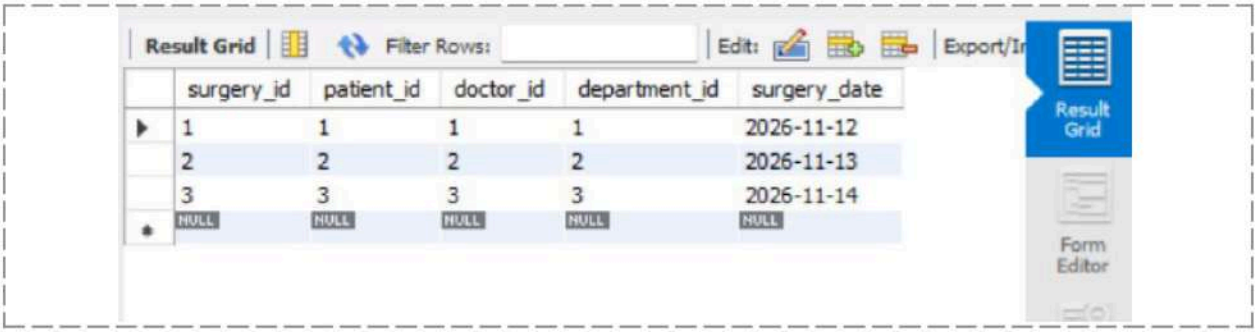
| history_id | patient_id | diagnosis | history_date |
|---|---|---|---|
| 1 | 1 | Heart Disease | 2026-11-01 |
| 2 | 2 | Migraine | 2026-11-02 |
| 3 | 3 | Bone Fracture | 2026-11-03 |
| 4 | 4 | Fever | 2026-11-04 |
| 5 | 5 | Ear Infection | 2026-11-05 |
| NULL | NULL | NULL | NULL |



Result Grid | Filter Rows: | Edit: | Export/Import

| treatment_id | patient_id | doctor_id | treatment_details |
|---|---|---|---|
| 1 | 1 | 1 | Cardiac Monitoring |
| 2 | 2 | 2 | Migraine Medication |
| 3 | 3 | 3 | Fracture Treatment |
| 4 | 4 | 4 | Fever Medication |
| 5 | 5 | 5 | Ear Infection Treatment |
| NULL | NULL | NULL | NULL |





Result Grid | Filter Rows: | Edit: | Export/Import

| surgery_id | patient_id | doctor_id | department_id | surgery_date |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2026-11-12 |
| 2 | 2 | 2 | 2 | 2026-11-13 |
| 3 | 3 | 3 | 3 | 2026-11-14 |
| NULL | NULL | NULL | NULL | NULL |



### 4. Explanation

SELECT * returns all columns and all rows of a table. Running it on every table confirms the inserted data: 5 departments, 5 doctors, 5 patients, 5 beds, 5 appointments, 4 admissions, 5 history records, 5 treatments and 3 surgeries.

## Query 25: Count Records in Each Table

### 1. Objective

To count the total number of records stored in every table.

### 2. SQL Query

*1. Departments*

```
SELECT COUNT(*) AS Total_Departments FROM Department;
```

*2. Doctors*

```
SELECT COUNT(*) AS Total_Doctors FROM Doctor;
```

*3. Patients*

```
SELECT COUNT(*) AS Total_Patients FROM Patients;
```

*4. Beds*

```
SELECT COUNT(*) AS Total_Beds FROM Bed;
```

*5. Appointments*

```
SELECT COUNT(*) AS Total_Appointments FROM Appointment;
```

*6. Admissions*

```
SELECT COUNT(*) AS Total_Admissions FROM Admission;
```

*7. Medical history*

```
SELECT COUNT(*) AS Total_History FROM Medical_History;
```

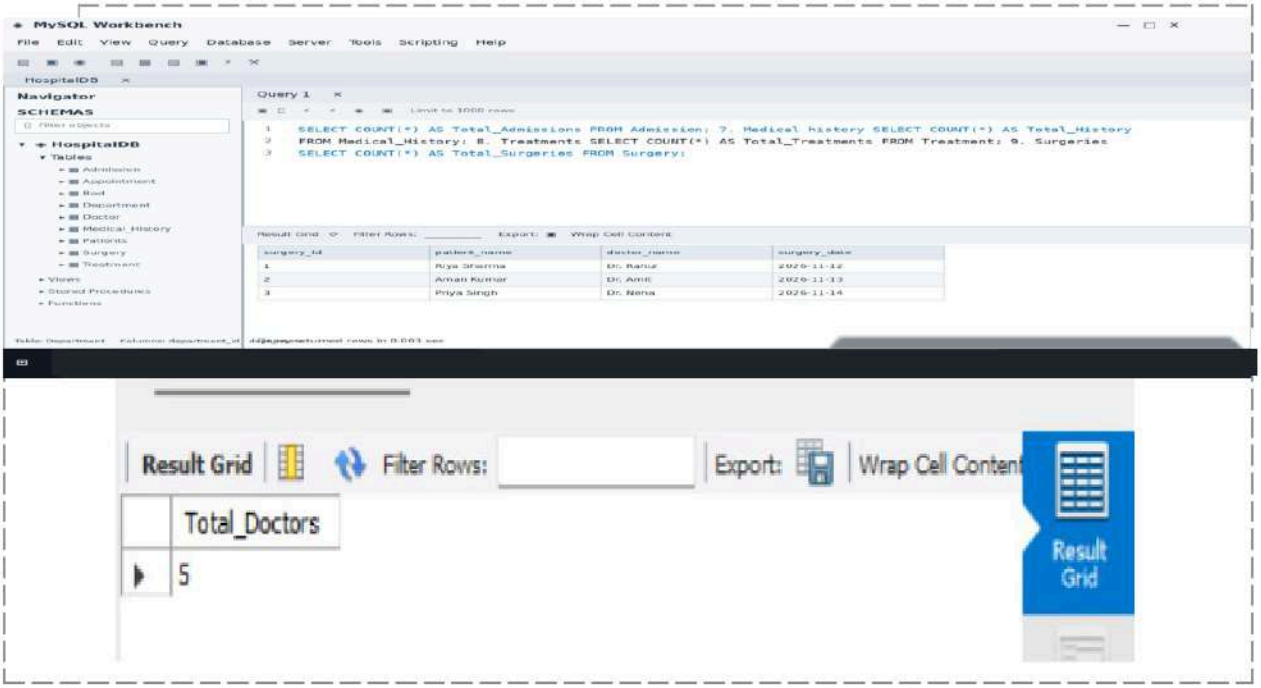
*8. Treatments*

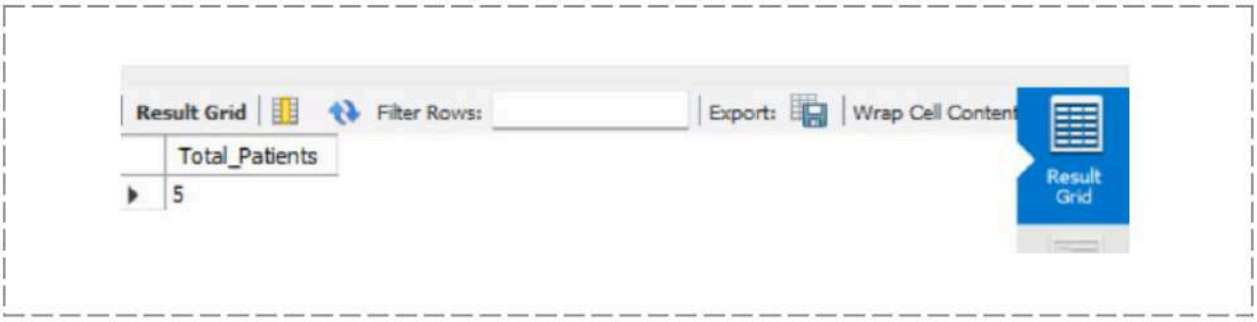
```
SELECT COUNT(*) AS Total_Treatments FROM Treatment;
```

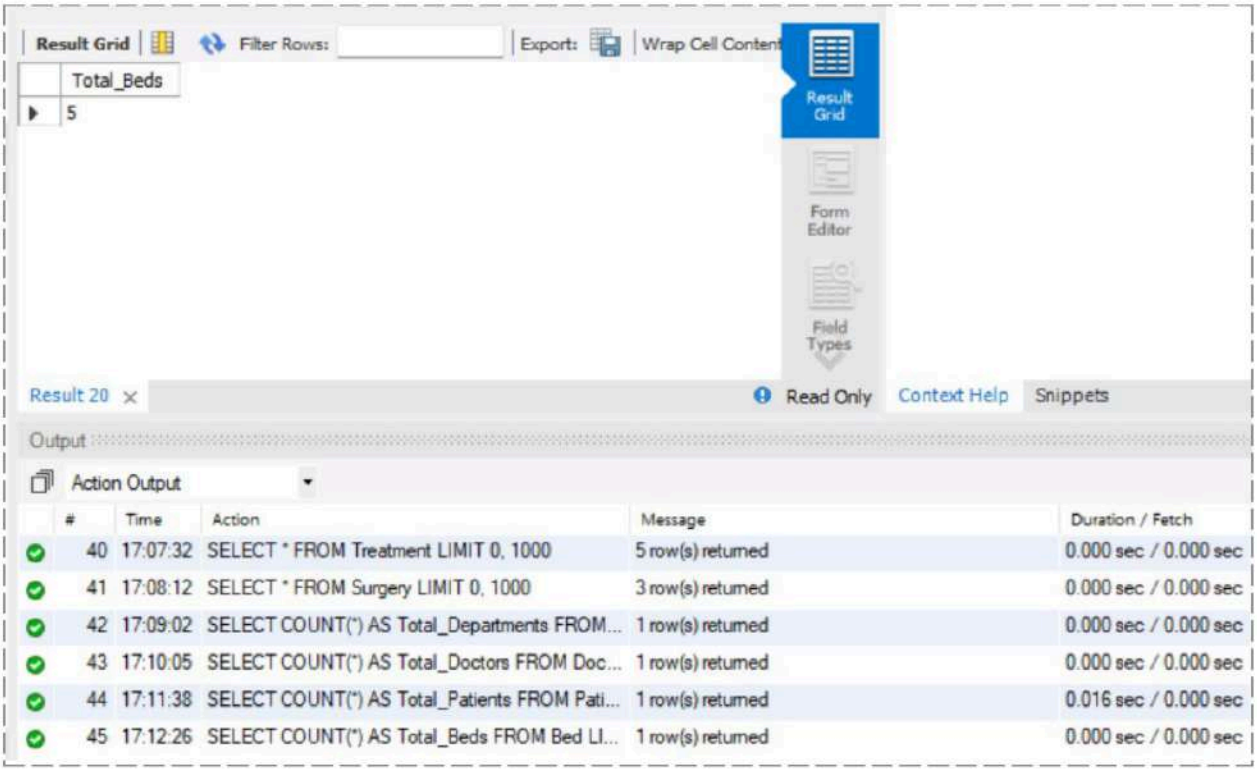
*9. Surgeries*

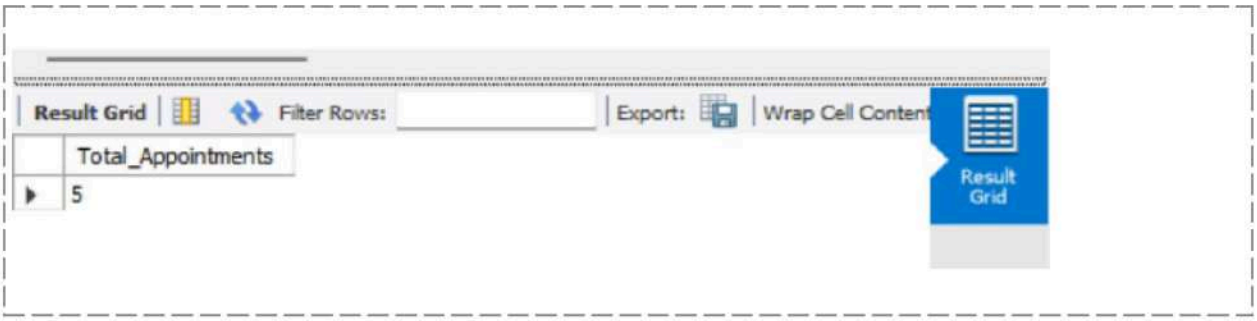
```
SELECT COUNT(*) AS Total_Surgeries FROM Surgery;
```

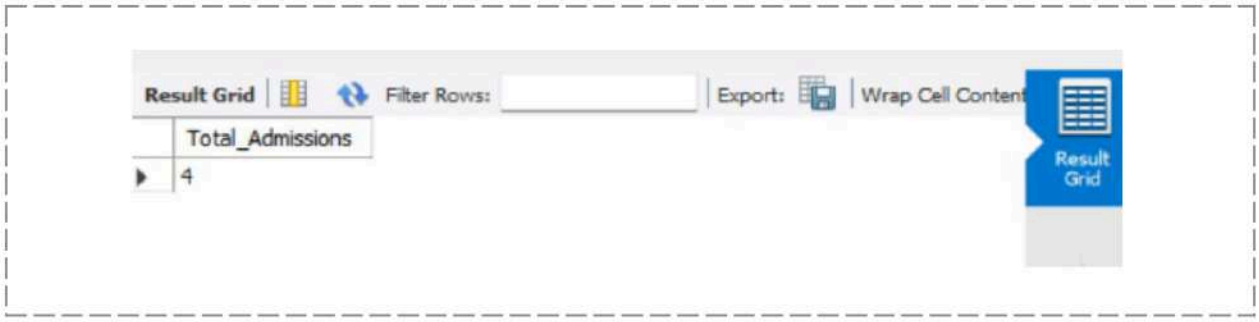
## 3. Execution Screenshot

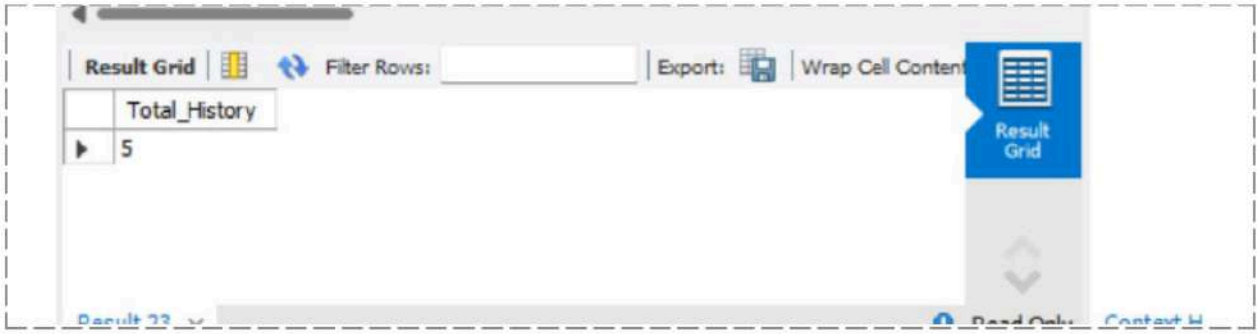



Result Grid
Filter Rows:
Export:
Wrap Cell Content
Total_Patients
5
Result Grid

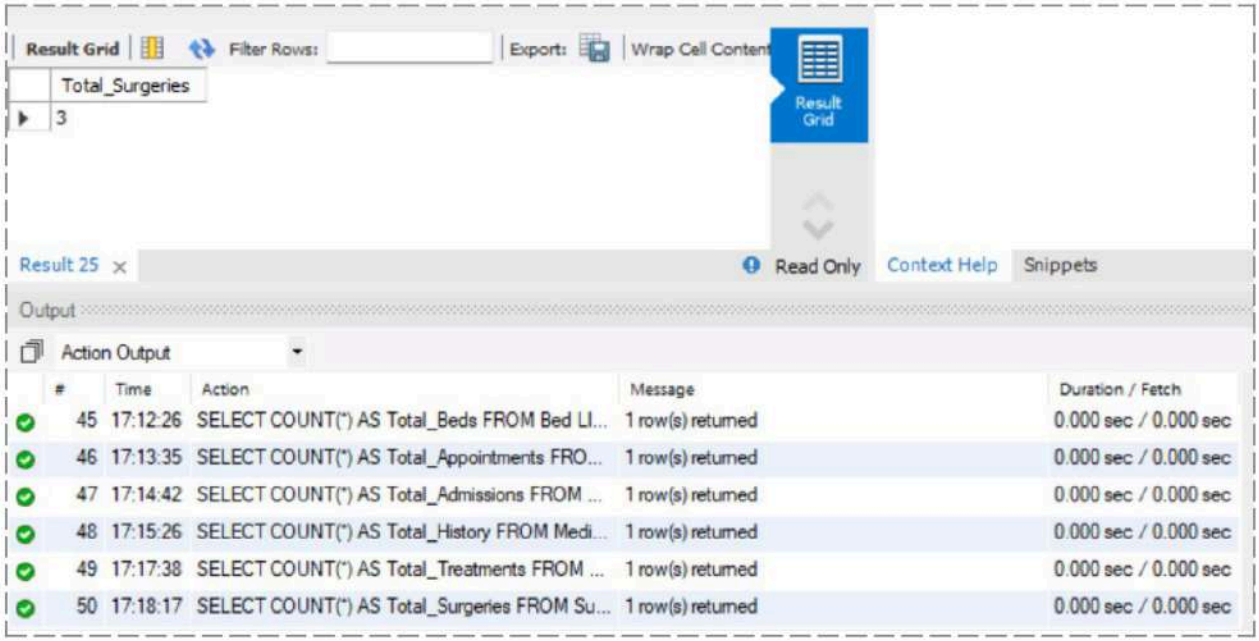

Result Grid
Filter Rows:
Export:
Wrap Cell Content
Total_Beds
5
Result Grid
Form Editor
Field Types
Result 20
Read Only
Context Help
Snippets
Output
Action Output
# Time Action Message Duration / Fetch
40 17:07:32 SELECT * FROM Treatment LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec
41 17:08:12 SELECT * FROM Surgery LIMIT 0, 1000 3 row(s) returned 0.000 sec / 0.000 sec
42 17:09:02 SELECT COUNT(*) AS Total_Departments FROM... 1 row(s) returned 0.000 sec / 0.000 sec
43 17:10:05 SELECT COUNT(*) AS Total_Doctors FROM Doc... 1 row(s) returned 0.000 sec / 0.000 sec
44 17:11:38 SELECT COUNT(*) AS Total_Patients FROM Pati... 1 row(s) returned 0.016 sec / 0.000 sec
45 17:12:26 SELECT COUNT(*) AS Total_Beds FROM Bed LI... 1 row(s) returned 0.000 sec / 0.000 sec


Result Grid
Filter Rows:
Export:
Wrap Cell Content
Total_Appointments
5
Result Grid


Result Grid
Filter Rows:
Export:
Wrap Cell Content
Result Grid
Total_Admissions
4


Result Grid
Filter Rows:
Export:
Wrap Cell Content
Result Grid
Total_History
5


Result Grid
Filter Rows:
Export:
Wrap Cell Content
Result Grid
Total_Treatments
5


Result Grid
Filter Rows:
Export:
Wrap Cell Content
Result Grid
Total_Surgeries
3
Result 25
Read Only
Context Help
Snippets
Output
Action Output
# Time Action Message Duration / Fetch
45 17:12:26 SELECT COUNT(*) AS Total_Beds FROM Bed LI... 1 row(s) returned 0.000 sec / 0.000 sec
46 17:13:35 SELECT COUNT(*) AS Total_Appointments FRO... 1 row(s) returned 0.000 sec / 0.000 sec
47 17:14:42 SELECT COUNT(*) AS Total_Admissions FROM ... 1 row(s) returned 0.000 sec / 0.000 sec
48 17:15:26 SELECT COUNT(*) AS Total_History FROM Medi... 1 row(s) returned 0.000 sec / 0.000 sec
49 17:17:38 SELECT COUNT(*) AS Total_Treatments FROM ... 1 row(s) returned 0.000 sec / 0.000 sec
50 17:18:17 SELECT COUNT(*) AS Total_Surgeries FROM Su... 1 row(s) returned 0.000 sec / 0.000 sec

### 4. Explanation

COUNT(*) is an aggregate function that counts the rows of a table and the alias (AS) gives a readable column name. The counts give a quick summary of the hospital data: 5 departments, 5 doctors, 5 patients, 5 beds, 5 appointments, 4 admissions, 5 history records, 5 treatments and 3 surgeries.

## Query 26: Retrieve Female Patients

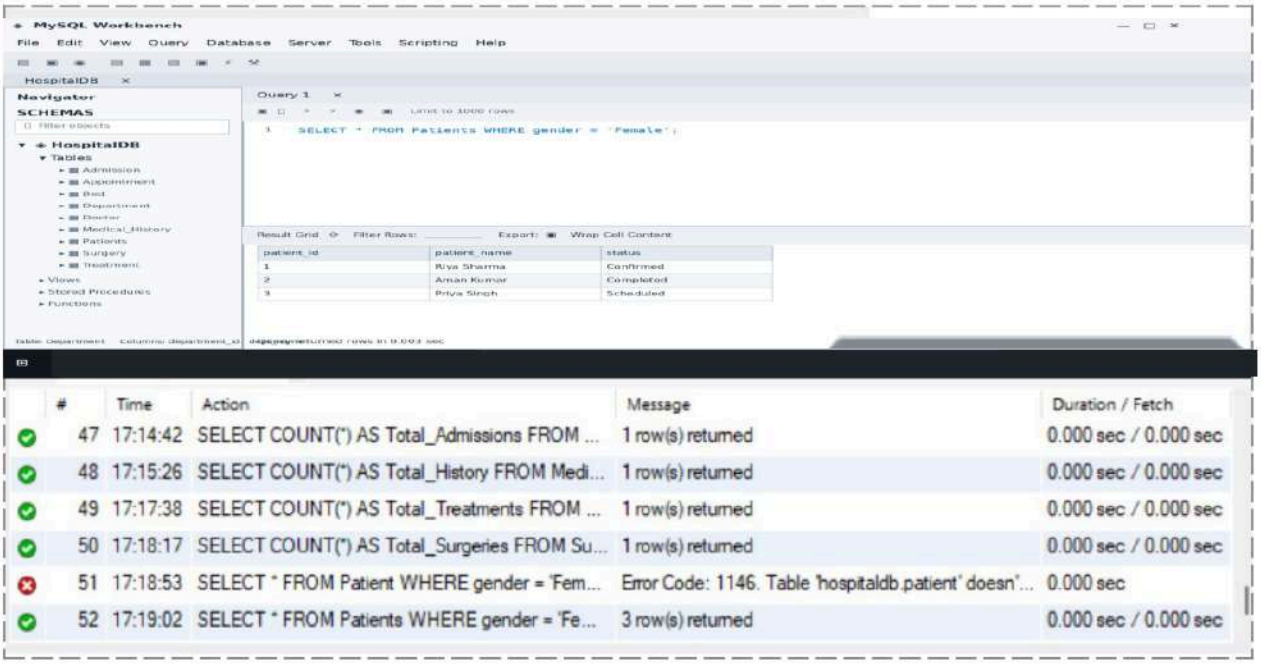
### 1. Objective

To display all the female patients of the hospital.

### 2. SQL Query

```
SELECT * FROM Patients
WHERE gender = 'Female';
```

### 3. Execution Screenshot




### 4. Explanation

The WHERE clause filters the rows and keeps only those where gender is 'Female'. The query returns three patients: Riya Sharma, Priya Singh and Sneha Gupta. The WHERE clause is evaluated for each row and keeps only records whose gender value exactly matches Female. Text comparisons can depend on the column collation, which may be case-insensitive by default in MySQL. The query demonstrates row filtering; it does not change or delete any patient data.

## Query 27: Doctors with Their Departments (INNER JOIN)

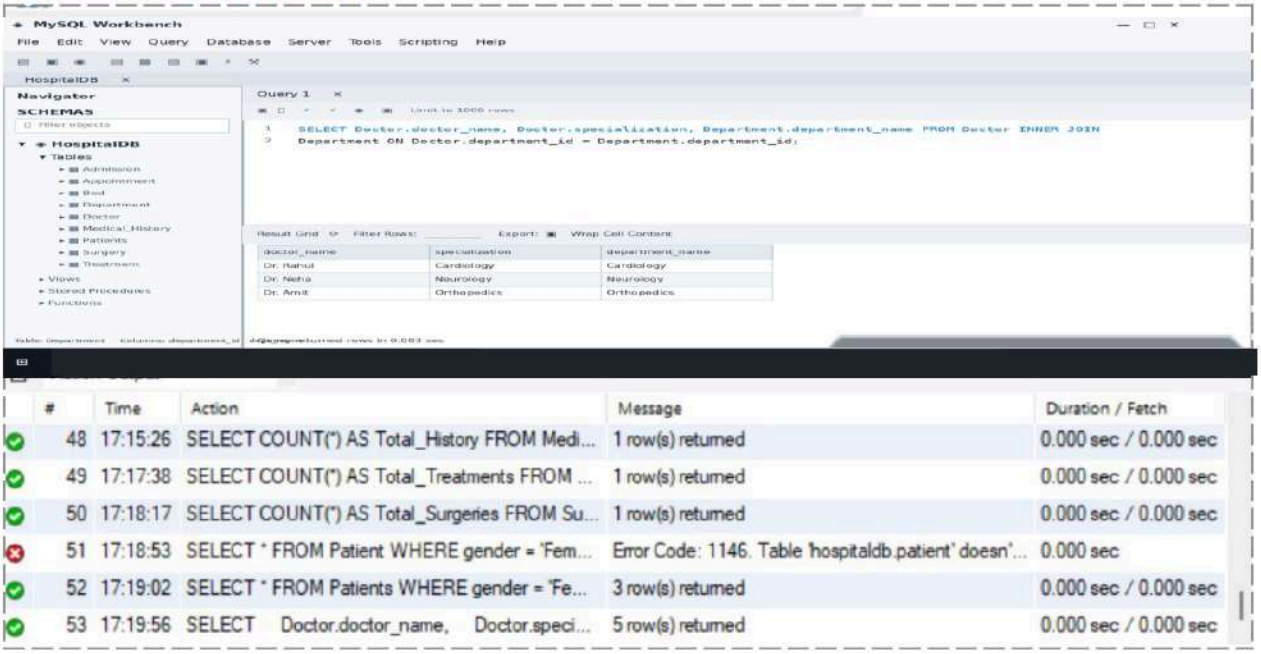
### 1. Objective

To display each doctor along with the name of the department in which he or she works.

### 2. SQL Query

```
SELECT
    Doctor.doctor_name,
    Doctor.specialization,
    Department.department_name
FROM Doctor
INNER JOIN Department
ON Doctor.department_id = Department.department_id;
```

### 3. Execution Screenshot




### 4. Explanation

INNER JOIN combines the Doctor and Department tables using the common column department_id. Only the matching rows are returned, so the result shows each of the five doctors with his or her specialization and department name. INNER JOIN returns only rows for which the join condition matches in both tables. The department_id is the shared key, and qualifying the column names with table names avoids ambiguity. If a doctor had no matching department, that doctor would not appear in this result; a LEFT JOIN would be needed to retain all doctors.

## Query 28: Appointment Details Using Multiple Joins

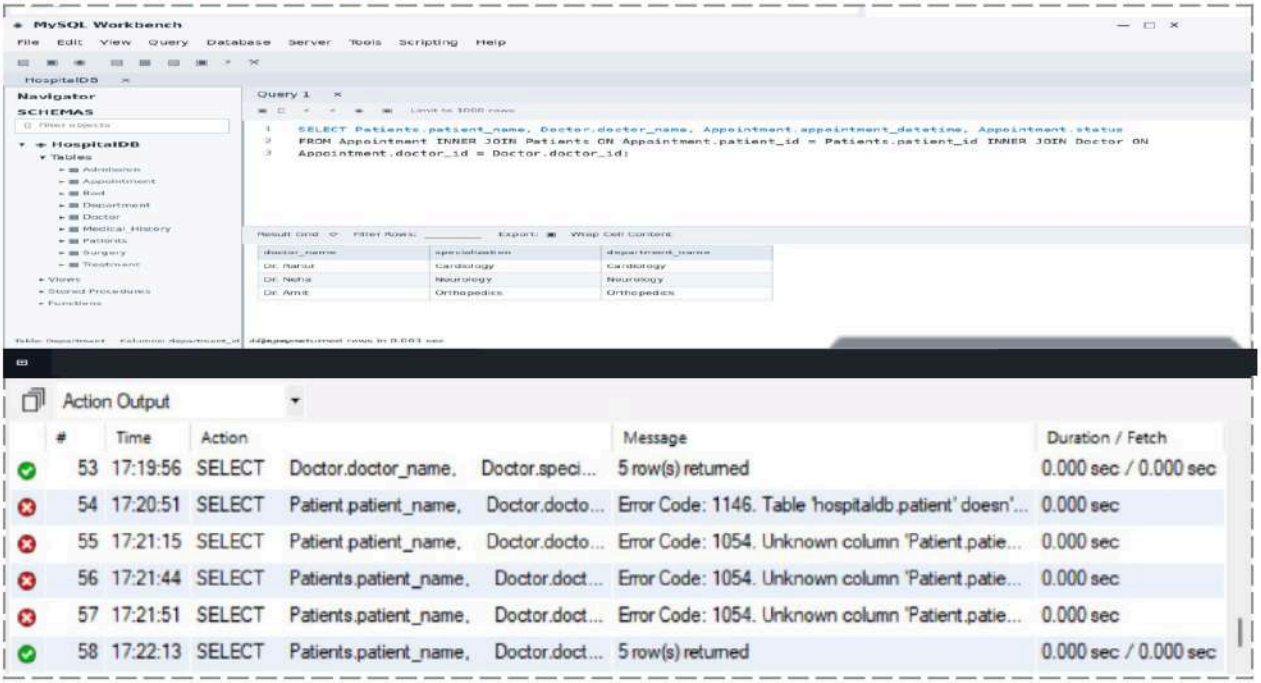
### 1. Objective

To display the patient name, doctor name, appointment date and time, and appointment status in one result.

### 2. SQL Query

```
SELECT
    Patients.patient_name,
    Doctor.doctor_name,
    Appointment.appointment_datetime,
    Appointment.status
FROM Appointment
INNER JOIN Patients
ON Appointment.patient_id = Patients.patient_id
INNER JOIN Doctor
ON Appointment.doctor_id = Doctor.doctor_id;
```

### 3. Execution Screenshot




### 4. Explanation

The Appointment table stores only the ids of the patient and the doctor. Two INNER JOINs replace these ids by the actual names from the Patient and Doctor tables, so the output is easy to read for the front-desk staff. The Appointment table contains identifiers, while Patients and Doctor contain readable names. Joining all three tables creates a report that staff can interpret without manually looking up IDs. The join conditions are essential: omitting one can create a Cartesian multiplication of rows and produce incorrect appointment totals.

## Query 29: Total Number of Patients

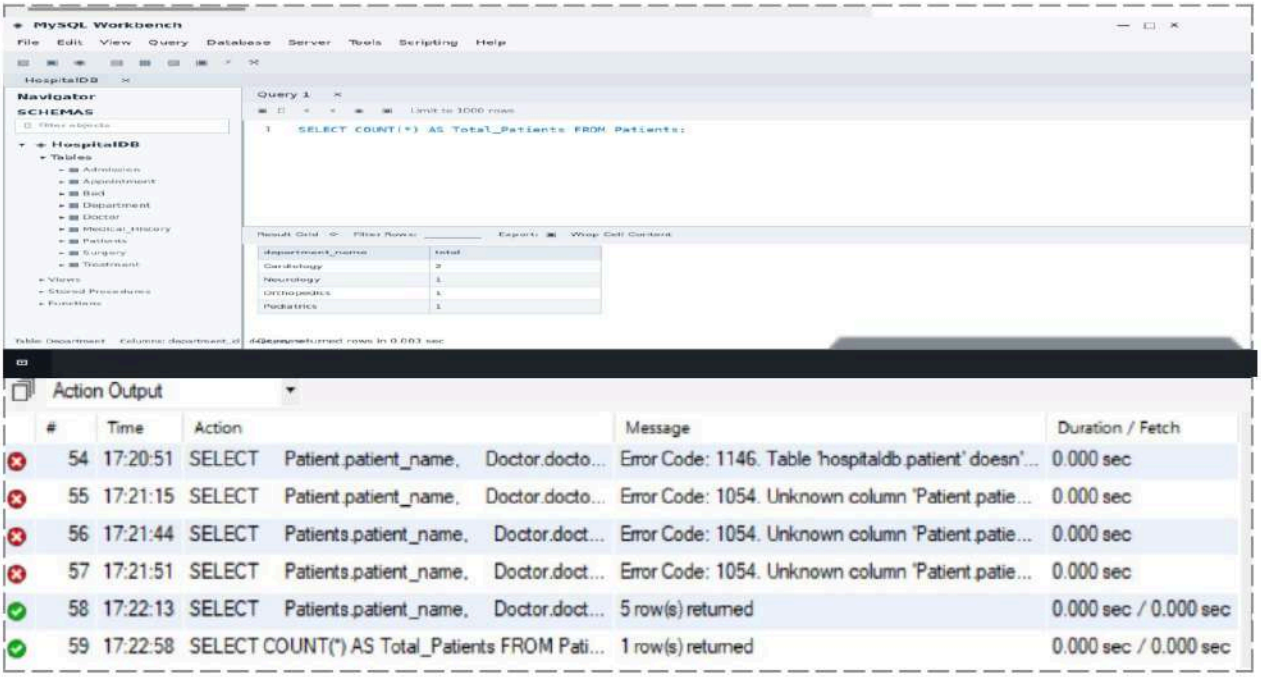
### 1. Objective

To find the total number of patients registered in the hospital.

### 2. SQL Query

```
SELECT COUNT(*) AS Total_Patients
FROM Patients;
```

### 3. Execution Screenshot




### 4. Explanation

COUNT(*) counts all the rows of the Patient table. The result is 5, which means five patients are registered. This count is a compact summary of registrations and can be used as a dashboard metric. COUNT(*) counts all patient rows, whereas COUNT(phone) would count only rows with a non-NULL phone value. The chosen expression therefore measures registered patient records, not the number of patients with complete contact details.

## Query 30: Number of Doctors in Each Department (LEFT JOIN with GROUP BY)

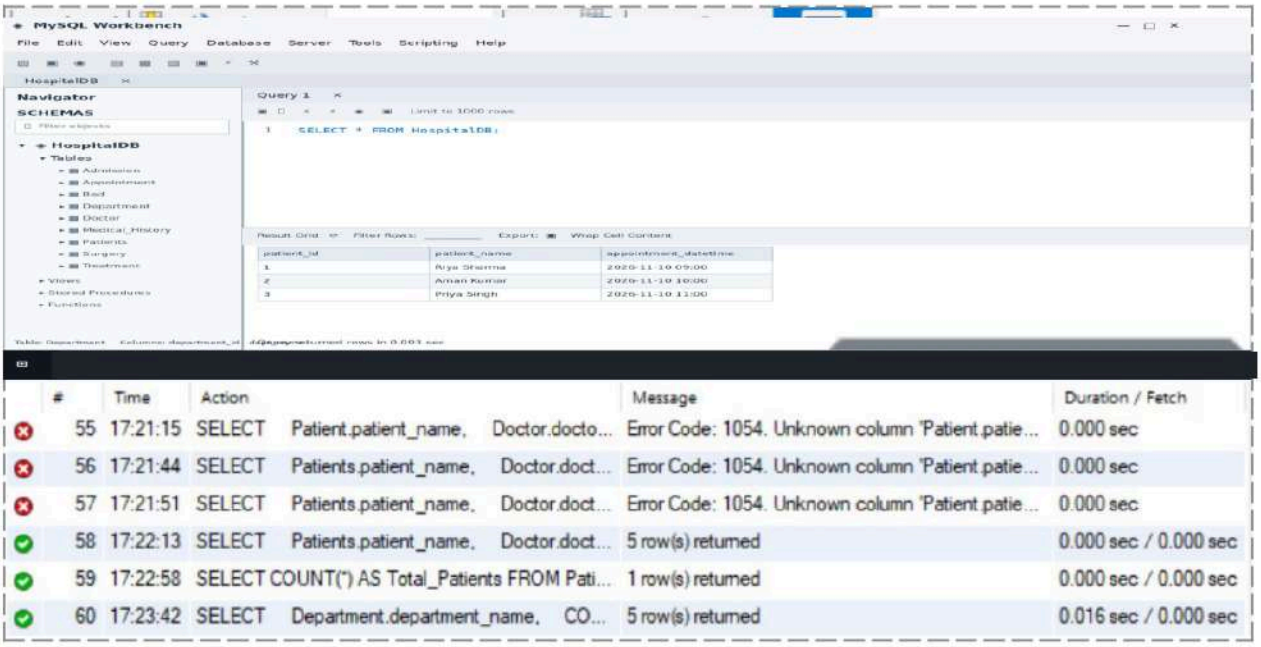
### 1. Objective

To count how many doctors work in each department, including the departments that have no doctor.

### 2. SQL Query

```
SELECT
    Department.department_name,
    COUNT(Doctor.doctor_id) AS Total_Doctors
FROM Department
LEFT JOIN Doctor
ON Department.department_id = Doctor.department_id
GROUP BY Department.department_name;
```

### 3. Execution Screenshot




### 4. Explanation

LEFT JOIN keeps every department even if no doctor is assigned to it, and GROUP BY forms one group for each department. COUNT(Doctor.doctor_id) counts the doctors in each group. Each of the five departments currently has one doctor. LEFT JOIN is selected because the report should retain a department even when no doctor is assigned. COUNT(Doctor.doctor_id) is used instead of COUNT(*) because the unmatched side of a LEFT JOIN contains NULL values; COUNT(column) ignores NULL and therefore returns zero for a department with no doctor.

## Query 31: Currently Admitted Patients

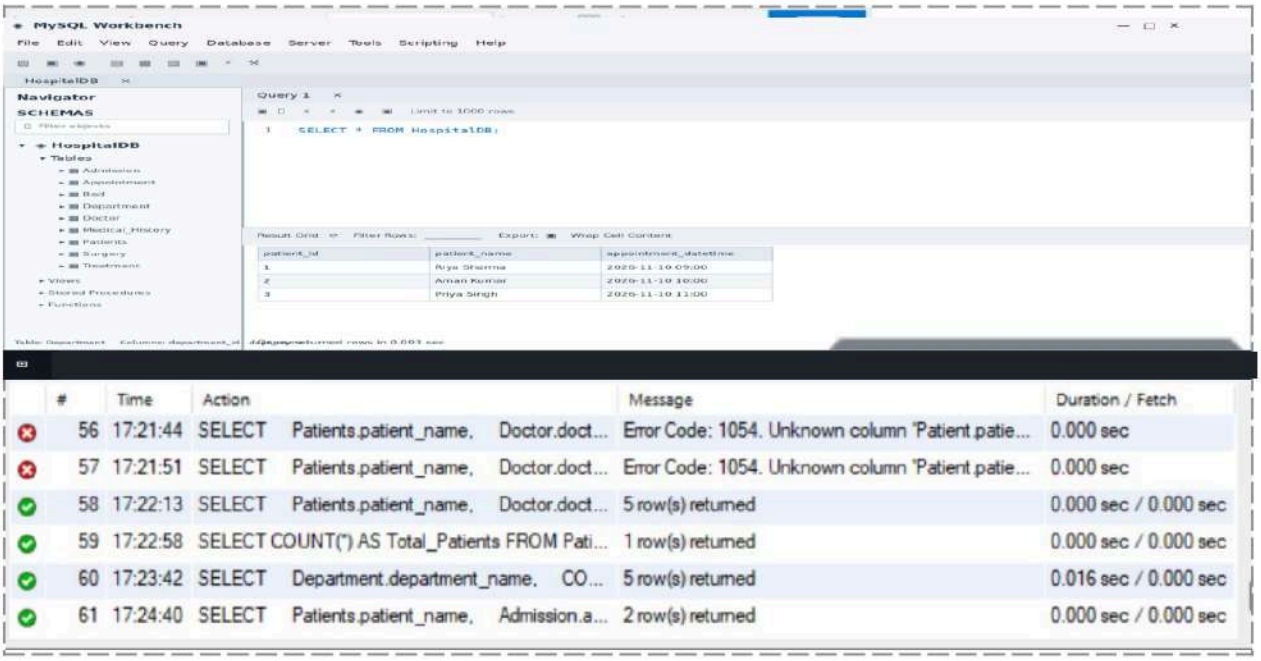
### 1. Objective

To list the patients who are still admitted in the hospital, together with their admission date and bed number.

### 2. SQL Query

```
SELECT
    Patients.patient_name,
    Admission.admission_date,
    Bed.bed_number
FROM Admission
INNER JOIN Patients
ON Admission.patient_id = Patients.patient_id
INNER JOIN Bed
ON Admission.bed_id = Bed.bed_id
WHERE Admission.discharge_date IS NULL;
```

### 3. Execution Screenshot




### 4. Explanation

The Admission table is joined with Patient and Bed. The condition discharge_date IS NULL keeps only the patients who have not been discharged yet. The result shows Riya Sharma in bed C102 (admitted on 2026-11-01) and Priya Singh in bed C101 (admitted on 2026-11-03). IS NULL is the correct test for a missing discharge date; comparing the column with = NULL would not work as expected in SQL. The joins add patient and bed details to the admission record. In the sample, two admissions remain active, but this query assumes the discharge_date field is maintained correctly.

## Query 32: Patient Medical History

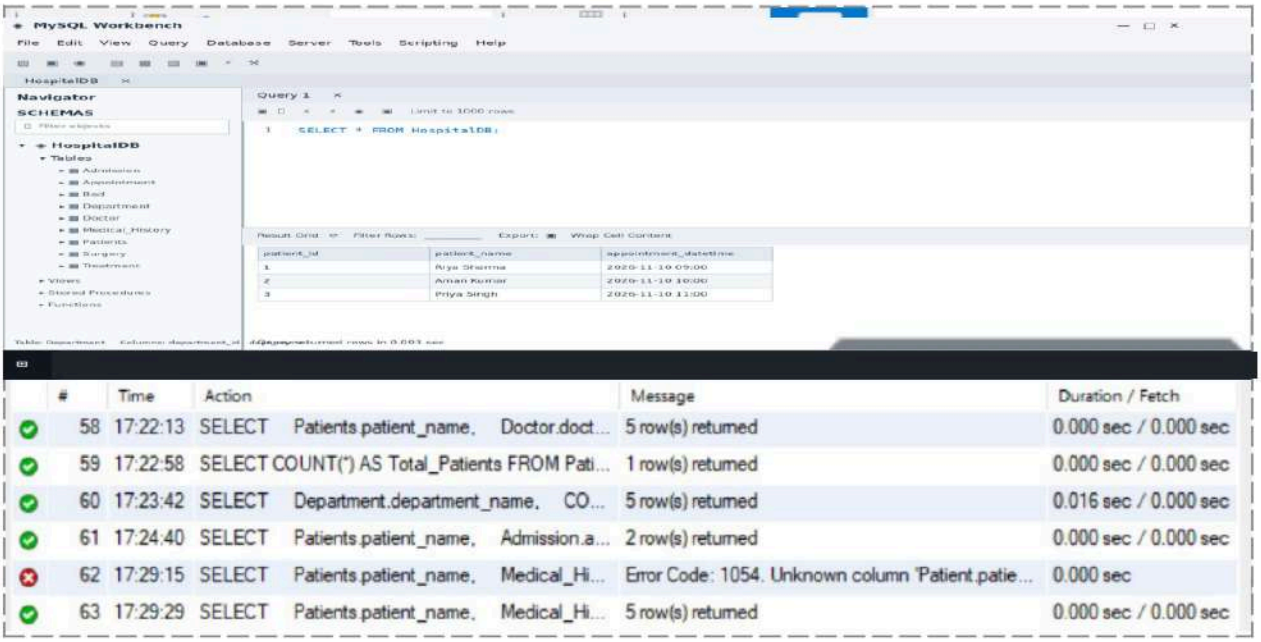
### 1. Objective

To display the diagnosis and diagnosis date of every patient.

### 2. SQL Query

```
SELECT
    Patients.patient_name,
    Medical_History.diagnosis,
    Medical_History.history_date
FROM Patients
INNER JOIN Medical_History
ON Patients.patient_id = Medical_History.patient_id;
```

### 3. Execution Screenshot




### 4. Explanation

The Patient and Medical_History tables are joined on patient_id so that every diagnosis is shown against the patient's name. This is the query used to retrieve the history of patients. The join connects every diagnosis record to the patient who owns it. If a patient has multiple medical-history rows, that patient will appear multiple times, once for each diagnosis. This is expected for a history report and should not be mistaken for duplicate patient records.

## Query 33: Appointments Grouped by Status

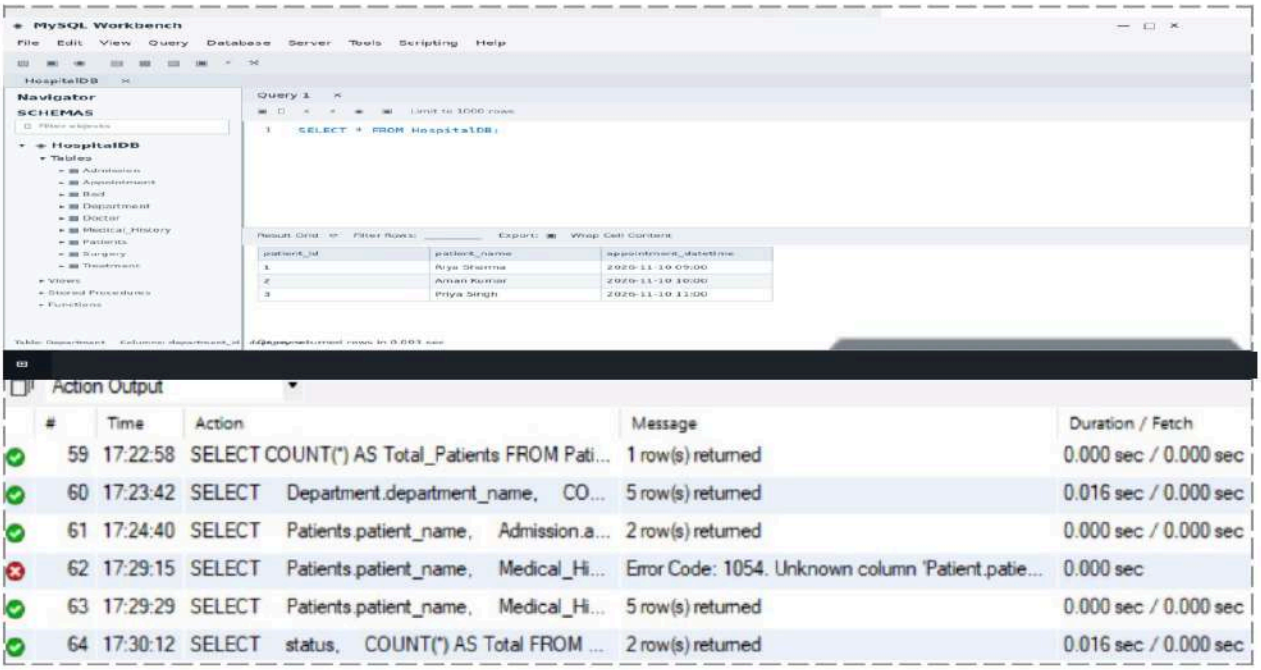
### 1. Objective

To count how many appointments are Scheduled and how many are Completed.

### 2. SQL Query

```
SELECT
    status,
    COUNT(*) AS Total
FROM Appointment
GROUP BY status;
```

### 3. Execution Screenshot




### 4. Explanation

GROUP BY status puts the appointments with the same status in one group and COUNT(*) counts them. The result shows 3 Scheduled and 2 Completed appointments. GROUP BY creates one result group for each distinct status, and COUNT(*) measures the number of appointments in each group. The sample result is three Scheduled and two Completed. If a new status such as Cancelled is inserted, it will automatically appear as another group.

## Query 34: Doctors Having Appointments (GROUP BY with HAVING)

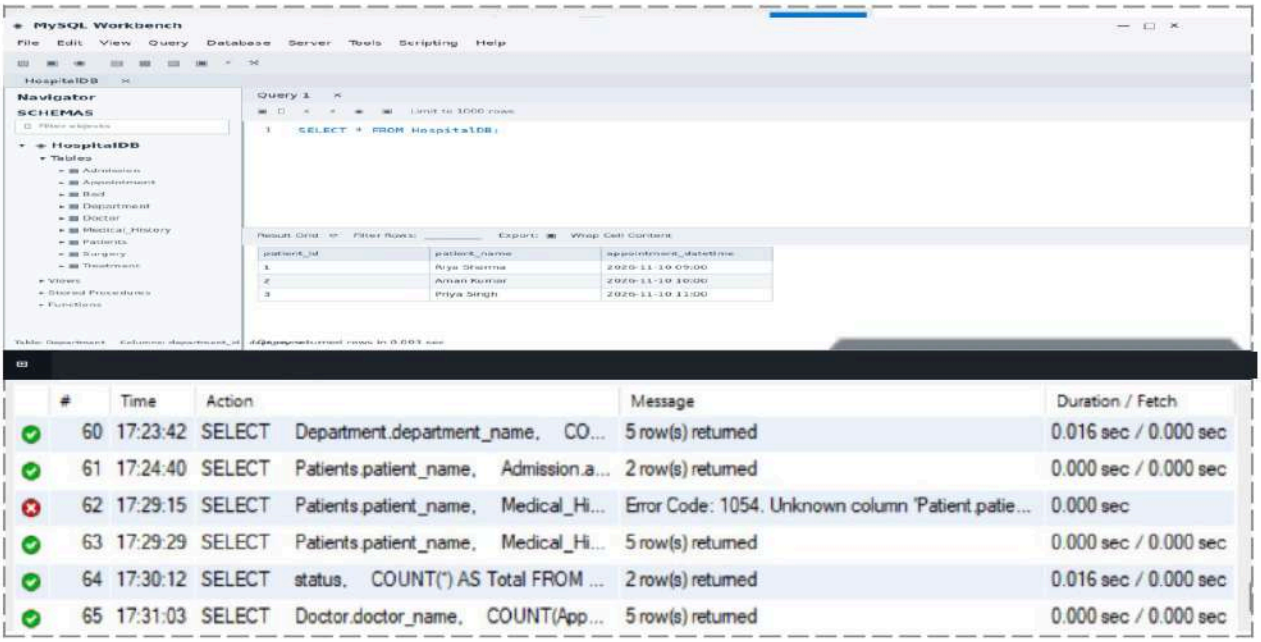
### 1. Objective

To display the number of appointments handled by each doctor, only for the doctors who have at least one appointment.

### 2. SQL Query

```
SELECT
    Doctor.doctor_name,
    COUNT(Appointment.appointment_id) AS Total_Appointments
FROM Doctor
LEFT JOIN Appointment
ON Doctor.doctor_id = Appointment.doctor_id
GROUP BY Doctor.doctor_id, Doctor.doctor_name
HAVING COUNT(Appointment.appointment_id) > 0;
```

### 3. Execution Screenshot




### 4. Explanation

The LEFT JOIN includes all doctors, GROUP BY counts the appointments of each doctor, and HAVING filters the groups after counting (WHERE cannot be used with an aggregate function). Every doctor has exactly one appointment, so all five doctors are displayed. WHERE filters individual rows before grouping, whereas HAVING filters the grouped results after COUNT has been calculated. The LEFT JOIN retains doctors without appointments, but the HAVING condition removes those with a count of zero. This distinction is central to writing aggregate reports correctly.

## Query 35: Patients Older than 25 (Date Function)

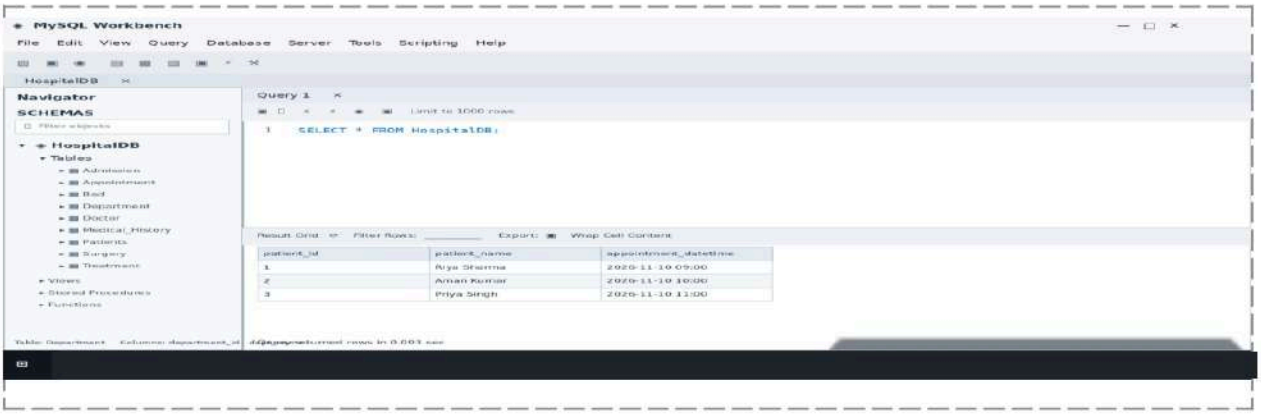
### 1. Objective

To retrieve the patients whose age is greater than 25 years.

### 2. SQL Query

```
SELECT *
FROM Patients
WHERE TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) > 25;
```

### 3. Execution Screenshot




### 4. Explanation

TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) calculates the age in completed years from the date of birth to the current date. The WHERE clause keeps the patients whose age is above 25. The result depends on the date on which the query is executed; in November 2026 it returns Aman Kumar, Priya Singh and Rahul Verma. TIMESTAMPDIFF with YEAR returns completed years between the date of birth and the current date, rather than simply subtracting the year numbers. CURDATE() makes the result change over time without editing the query. Because age is time-dependent, the result can differ when the query is run on another date.

## Query 36: Doctors Who Have Appointments (IN Subquery)

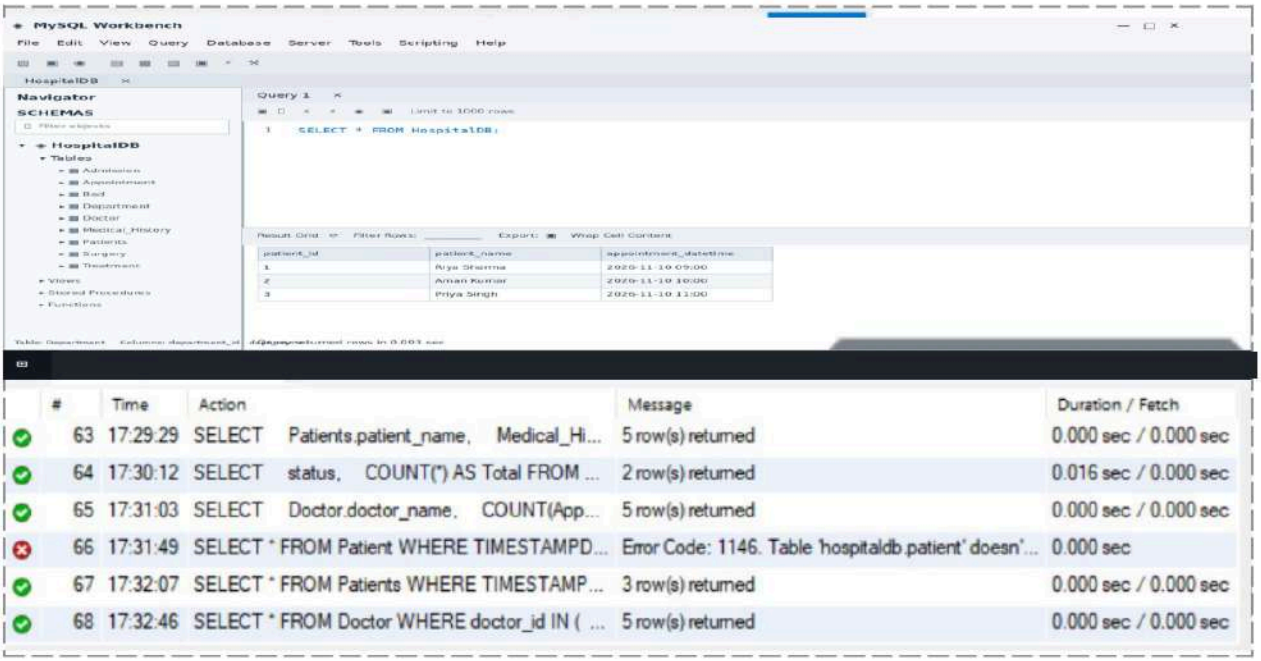
### 1. Objective

To display the doctors who have at least one appointment.

### 2. SQL Query

```
SELECT *
FROM Doctor
WHERE doctor_id IN (
    SELECT doctor_id
    FROM Appointment
);
```

### 3. Execution Screenshot




### 4. Explanation

The inner query returns the list of doctor ids present in the Appointment table and the outer query shows the doctors whose id is in that list. All five doctors have appointments, so all five rows are returned. The inner query produces the set of doctor IDs that appear in Appointment. The outer query then returns doctor records whose ID belongs to that set. IN is useful when the subquery can return multiple values; it expresses membership and avoids manually listing the doctor IDs.

## Query 37: Patients Who Have Appointments (IN Subquery)

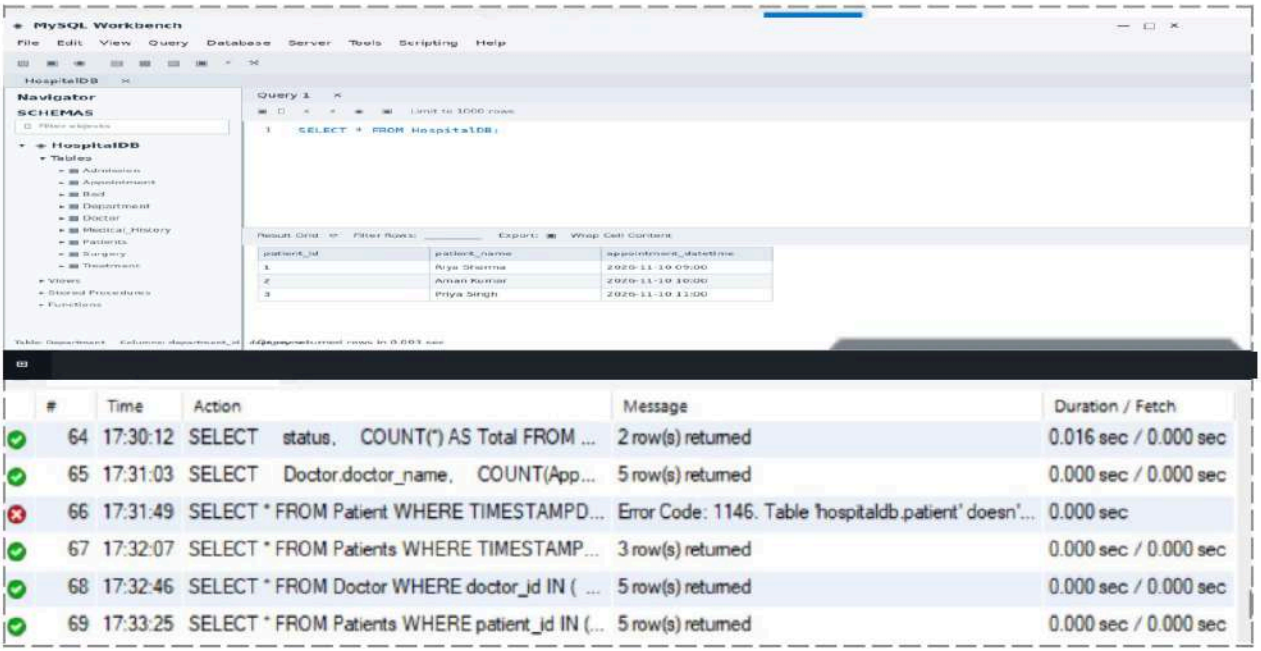
### 1. Objective

To display the patients who have booked at least one appointment.

### 2. SQL Query

```
SELECT *
FROM Patients
WHERE patient_id IN (
    SELECT patient_id
    FROM Appointment
);
```

### 3. Execution Screenshot




### 4. Explanation

The subquery collects the patient ids from the Appointment table and the outer query returns the matching patients. Every patient has an appointment, so all five patients are displayed. This is the patient-side version of the previous query. It identifies registered patients who have at least one appointment. A patient without an appointment is excluded, even though the patient remains present in the Patients table. This is useful for identifying active users of the appointment service.

## Query 38: Doctors with More Appointments than the Average (Nested Subquery)

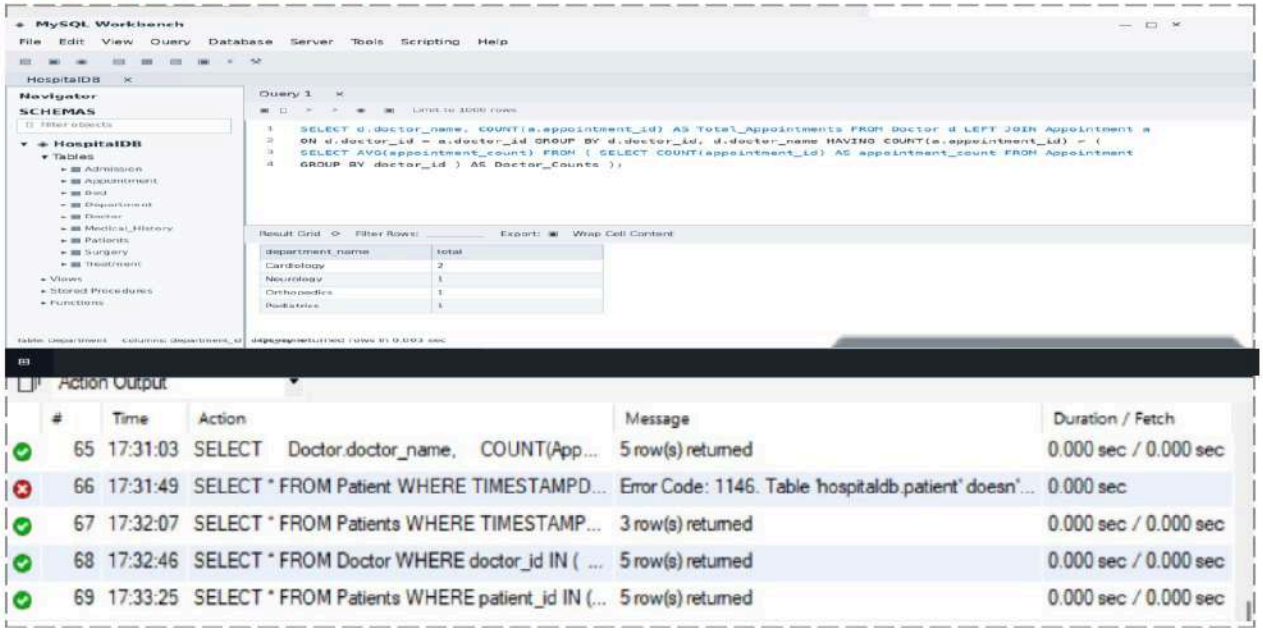
### 1. Objective

To find the doctors whose number of appointments is greater than the average number of appointments per doctor.

## 2. SQL Query

```
SELECT
    d.doctor_name,
    COUNT(a.appointment_id) AS Total_Appointments
FROM Doctor d
LEFT JOIN Appointment a
ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.doctor_name
HAVING COUNT(a.appointment_id) > (
    SELECT AVG(appointment_count)
    FROM (
        SELECT COUNT(appointment_id) AS appointment_count
        FROM Appointment
        GROUP BY doctor_id
    ) AS Doctor_Counts
);
```

## 3. Execution Screenshot




## 4. Explanation

The innermost query counts the appointments of every doctor, the middle query calculates the average of these counts, and the outer query keeps only the doctors whose count is above that average. In the present data every doctor has exactly one appointment, so the average is 1 and no doctor is above it; the query therefore returns an empty result. With more appointment data it will list the busiest doctors. The nested query first calculates appointment counts per doctor; the next level calculates the average of those counts; the outer query compares each doctor's count with that average. The sample has one appointment per doctor, so the average is one and no doctor is above it. The empty result is therefore a valid outcome, not a query failure.

## Query 39: Patients Who Have Never Been Admitted (NOT IN Subquery)

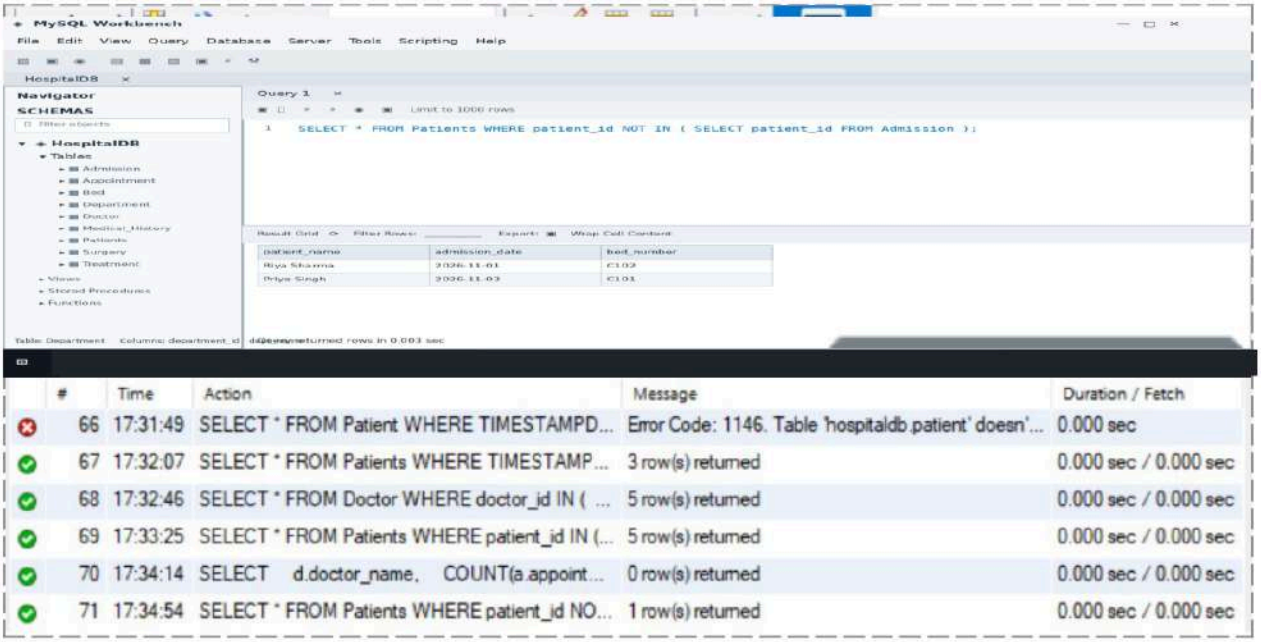
### 1. Objective

To find the patients who were never admitted to the hospital.

### 2. SQL Query

```
SELECT *
FROM Patients
WHERE patient_id NOT IN (
    SELECT patient_id
    FROM Admission
);
```

### 3. Execution Screenshot




### 4. Explanation

The subquery returns the ids of all admitted patients and NOT IN keeps the patients that are not in that list. Only Sneha Gupta has no admission record, so she is the only patient returned. NOT IN returns patients whose IDs are absent from the admission list. In this sample, Sneha Gupta has no admission record. In production SQL, NOT EXISTS is often preferred when the subquery might contain NULL, because NULL values can make NOT IN comparisons produce an unknown result.

## Query 40: Currently Admitted Patients (Hospital Report)

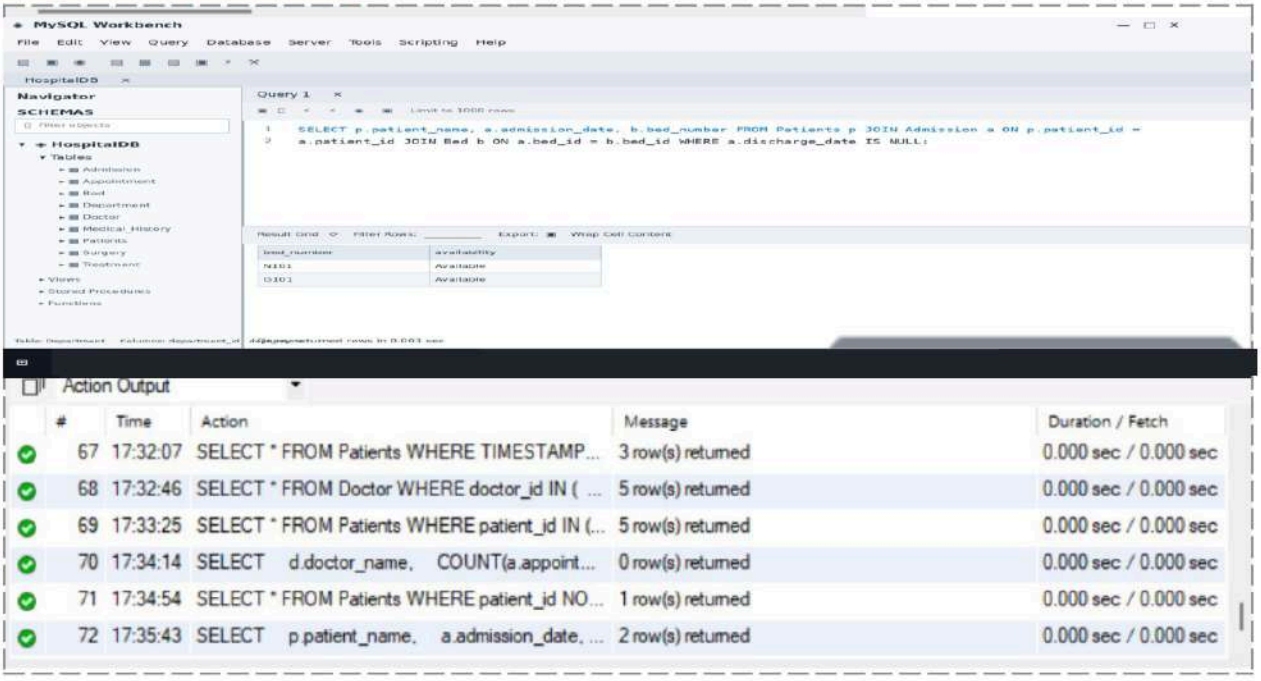
### 1. Objective

To generate the report of patients who are currently admitted, together with the bed they occupy.

### 2. SQL Query

```
SELECT
    p.patient_name,
    a.admission_date,
    b.bed_number
FROM Patients p
JOIN Admission a
ON p.patient_id = a.patient_id
JOIN Bed b
ON a.bed_id = b.bed_id
WHERE a.discharge_date IS NULL;
```

### 3. Execution Screenshot




### 4. Explanation

This is the same in-patient report written with table aliases (p, a and b), which makes the query shorter. It tracks patient admissions: a NULL discharge_date marks a patient who is still in the hospital. Table aliases shorten the SQL and make multi-table queries easier to read. This report intentionally repeats the current-patient logic from Query 31 in a compact form, showing that the same business question can be expressed with aliases. The query is read-only and does not update bed availability.

## Query 41: Available Beds by Department

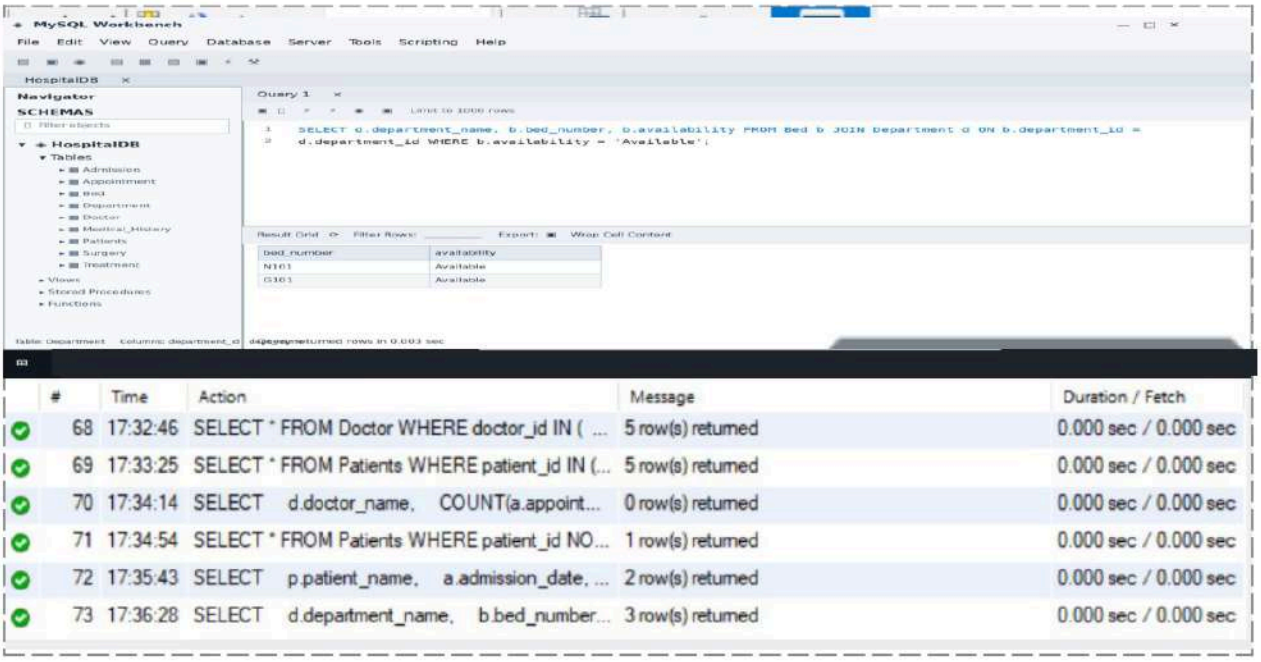
### 1. Objective

To manage hospital resources by listing all beds that are currently available, department-wise.

### 2. SQL Query

```
SELECT
    d.department_name,
    b.bed_number,
    b.availability
FROM Bed b
JOIN Department d
ON b.department_id = d.department_id
WHERE b.availability = 'Available';
```

### 3. Execution Screenshot




### 4. Explanation

The Bed table is joined with Department to display the department name for every bed, and the WHERE clause keeps only the beds with availability 'Available' .The bed query showed three available beds (N101, O101 and G101), which supports bed allotment. General Medicine, which helps the staff to allot a bed quickly. The query joins Bed to Department so that each available bed is shown with its department name. According to the INSERT statements, only N101 and G101 have availability='Available'; C101, C102 and O101 are marked Occupied. Therefore the correct sample result is two available beds, not three. This report relies on the stored availability field and does not independently verify active admissions.

## Query 42: Surgery Schedule

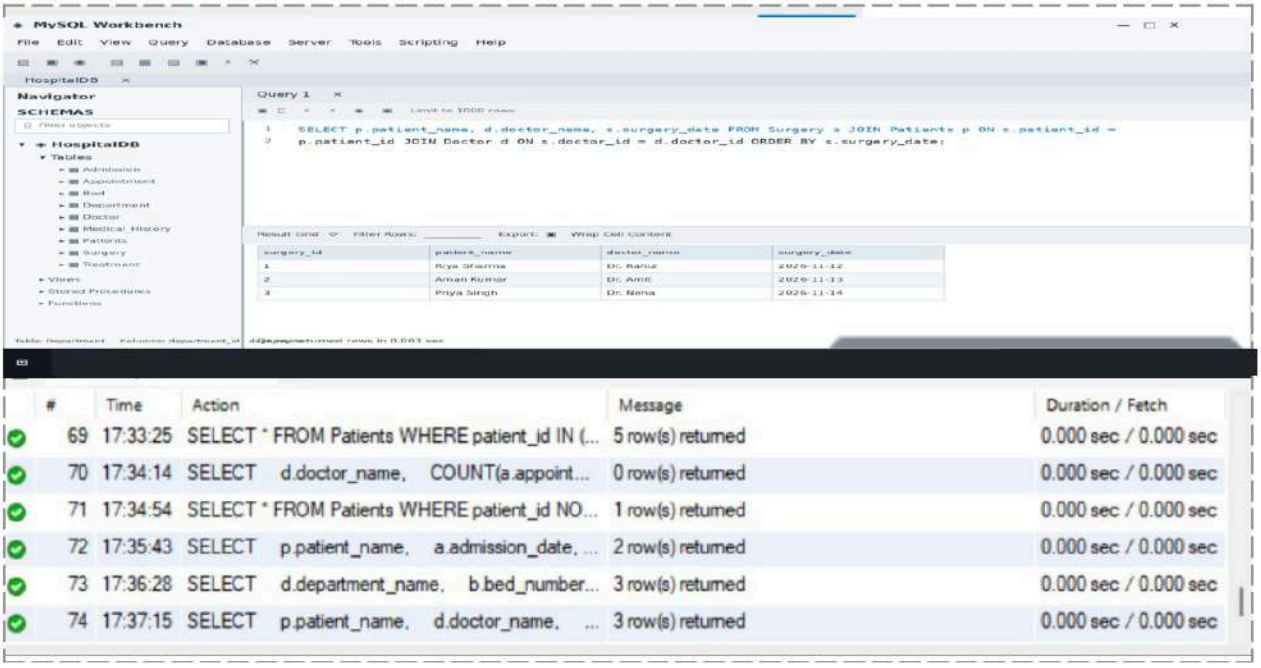
### 1. Objective

To display the surgery schedule with the patient, the operating doctor and the date, in date order.

### 2. SQL Query

```
SELECT
    p.patient_name,
    d.doctor_name,
    s.surgery_date
FROM Surgery s
JOIN Patients p
ON s.patient_id = p.patient_id
JOIN Doctor d
ON s.doctor_id = d.doctor_id
ORDER BY s.surgery_date;
```

### 3. Execution Screenshot




### 4. Explanation

The Surgery table is joined with Patient and Doctor, and ORDER BY sorts the result by surgery_date in ascending order. The schedule is: Riya Sharma with Dr. Rahul on 2026-11-12, Aman Kumar with Dr. Amit on 2026-11-13 and Priya Singh with Dr. Neha on 2026-11-14. The joins translate patient_id and doctor_id into names, while ORDER BY places the earliest surgery first. The sample schedule contains three procedures on 12, 13 and 14 November 2026. A complete operating-room scheduler would additionally consider time slots, duration, theatre capacity and doctor availability.

## Query 43: Department-wise Admissions

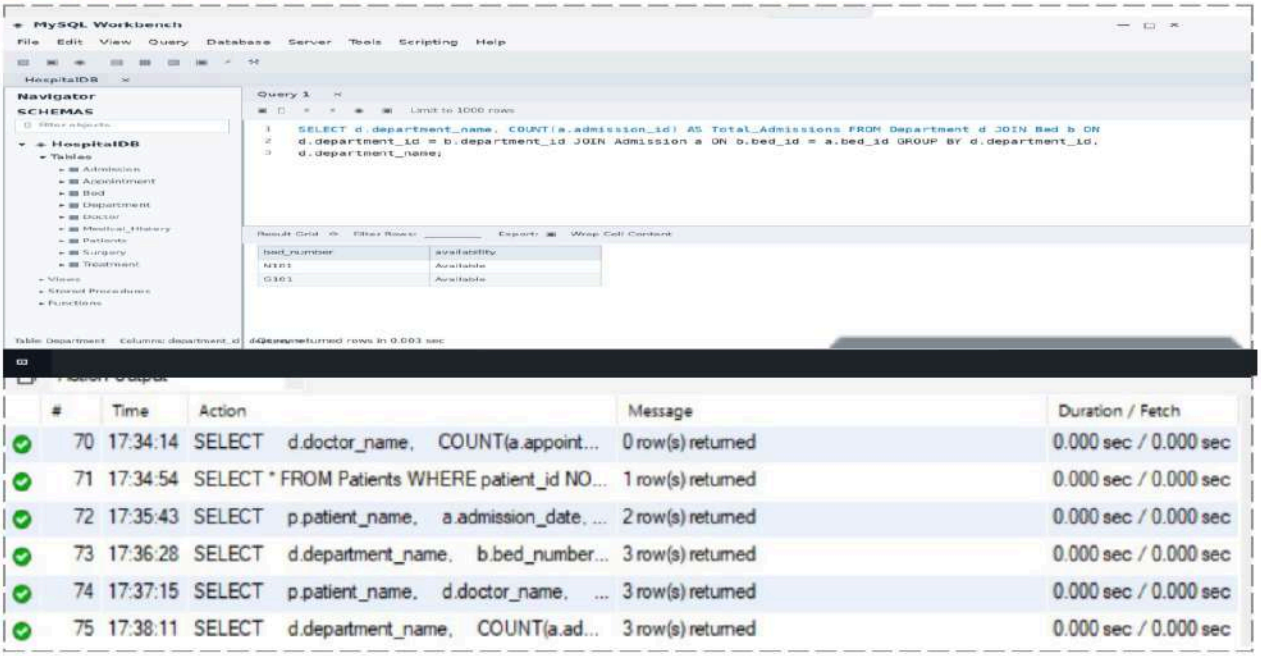
### 1. Objective

To count the total number of admissions in each department.

## 2. SQL Query

```
SELECT
    d.department_name,
    COUNT(a.admission_id) AS Total_Admissions
FROM Department d
JOIN Bed b
ON d.department_id = b.department_id
JOIN Admission a
ON b.bed_id = a.bed_id
GROUP BY d.department_id, d.department_name;
```

## 3. Execution Screenshot




## 4. Explanation

Admission does not store the department directly, so the query goes from Department to Bed and then to Admission using joins and groups the result by department. Cardiology has 2 admissions, and Orthopedics and General Medicine have 1 admission each. Neurology and ENT are not shown because they have no admission. Admission has no department_id column, so the query follows the relationship through Bed: Department to Bed, then Bed to Admission. GROUP BY creates one total per department. Departments with no matching admission are absent because INNER JOIN is used; LEFT JOIN would be required to show them with a zero count.

# INDEXING AND QUERY OPTIMIZATION

An index is a separate data structure (a B-tree in MySQL InnoDB) that allows the database to find rows without scanning the complete table. In this project two composite indexes are created for the two most frequently used lookups of a hospital system: the history of a patient's admissions and the schedule of a doctor's appointments.

## Query 44: Create Indexes

### 1. Objective

To create indexes on Admission(patient_id, admission_date) and Appointment(doctor_id, appointment_datetime) so that patient history retrieval and doctor schedule checks become faster.

### 2. SQL Query

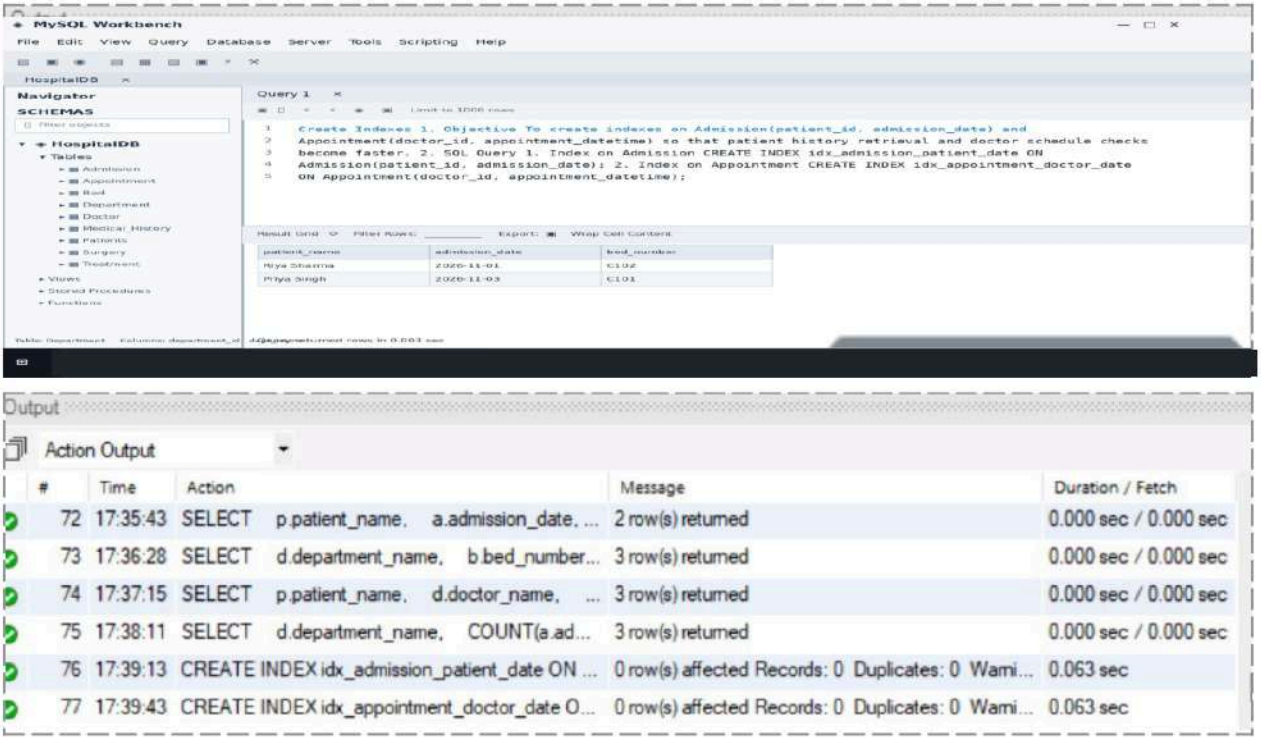
*1. Index on Admission*

```
CREATE INDEX idx_admission_patient_date
ON Admission(patient_id, admission_date);
```

*2. Index on Appointment*

```
CREATE INDEX idx_appointment_doctor_date
ON Appointment(doctor_id, appointment_datetime);
```

### 3. Execution Screenshot



### 4. Explanation

Both are composite indexes. idx_admission_patient_date sorts admissions first by patient_id and then by admission_date, so all the admissions of one patient are found quickly and already in date order. idx_appointment_doctor_date sorts appointments by doctor_id and then by appointment_datetime, which speeds up checking whether a doctor is free at a given time (schedule-conflict check). A composite index stores entries using the listed column order. The leftmost-prefix rule matters: an index on (patient_id, admission_date) can support searches by patient_id and by patient_id plus date, but it is not generally equivalent to an index beginning with admission_date. Indexes improve selected reads at the cost of storage and extra work during inserts, updates and deletes.

## Query 45: Show Index on Admission

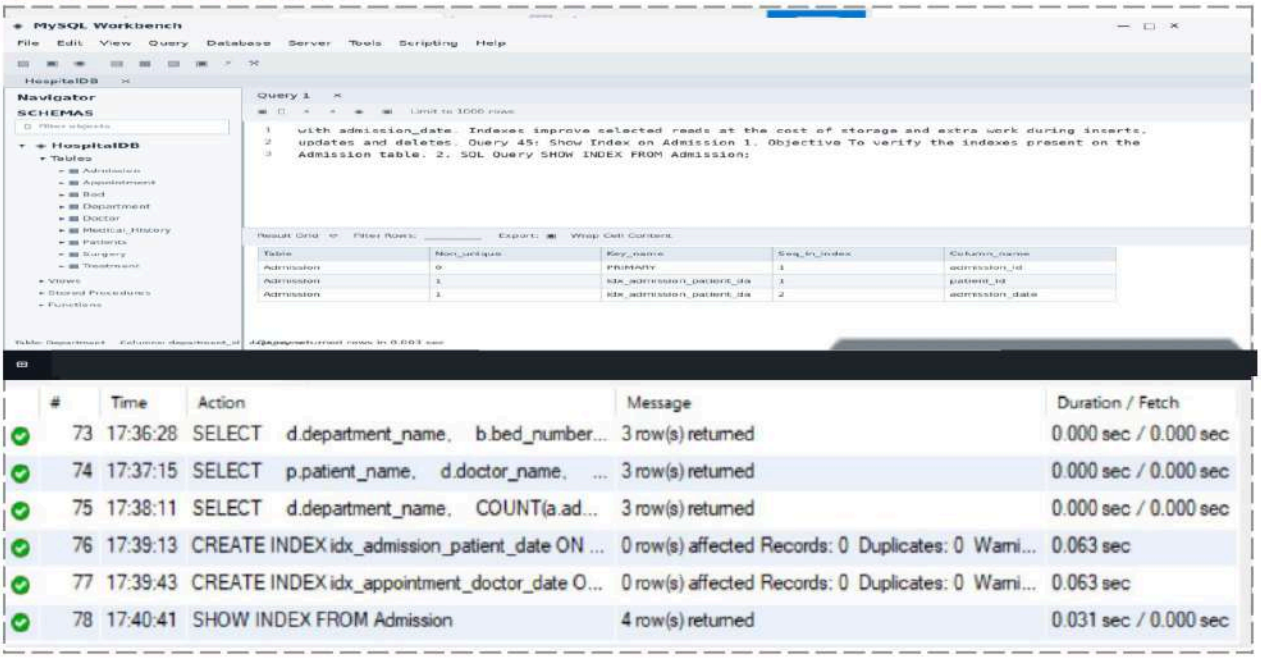
### 1. Objective

To verify the indexes present on the Admission table.

### 2. SQL Query

```
SHOW INDEX FROM Admission;
```

### 3. Execution Screenshot




### 4. Explanation

SHOW INDEX lists every index of the table with its columns and order. The output shows the primary key, the index automatically created for the foreign keys, and the new idx_admission_patient_date index with patient_id as the first column and admission_date as the second. SHOW INDEX exposes the index name, indexed column, sequence within a composite index and uniqueness information. Checking Seq_in_index confirms the order of the two columns. This is important because the column order determines which search conditions can benefit from the composite index.

## Query 46: Show Index on Appointment

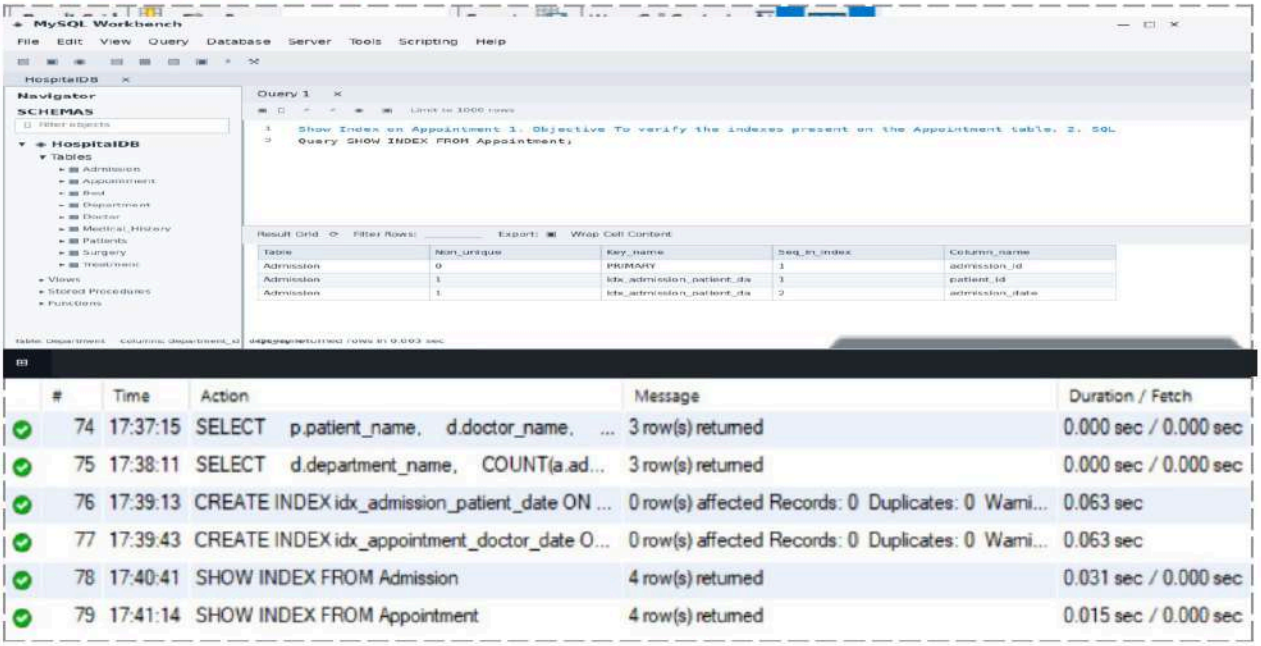
### 1. Objective

To verify the indexes present on the Appointment table.

### 2. SQL Query

```
SHOW INDEX FROM Appointment;
```

### 3. Execution Screenshot




### 4. Explanation

The output shows idx_appointment_doctor_date with doctor_id and appointment_datetime as its two columns, in addition to the primary key and the foreign-key indexes. This check confirms that doctor_id is the first indexed column and appointment_datetime is second. The index is aligned with queries that first restrict appointments to one doctor and then inspect dates or times. It should still be tested with realistic data because an index is not guaranteed to be selected for every query.

## Query 47: Analyse the Query Plan with EXPLAIN

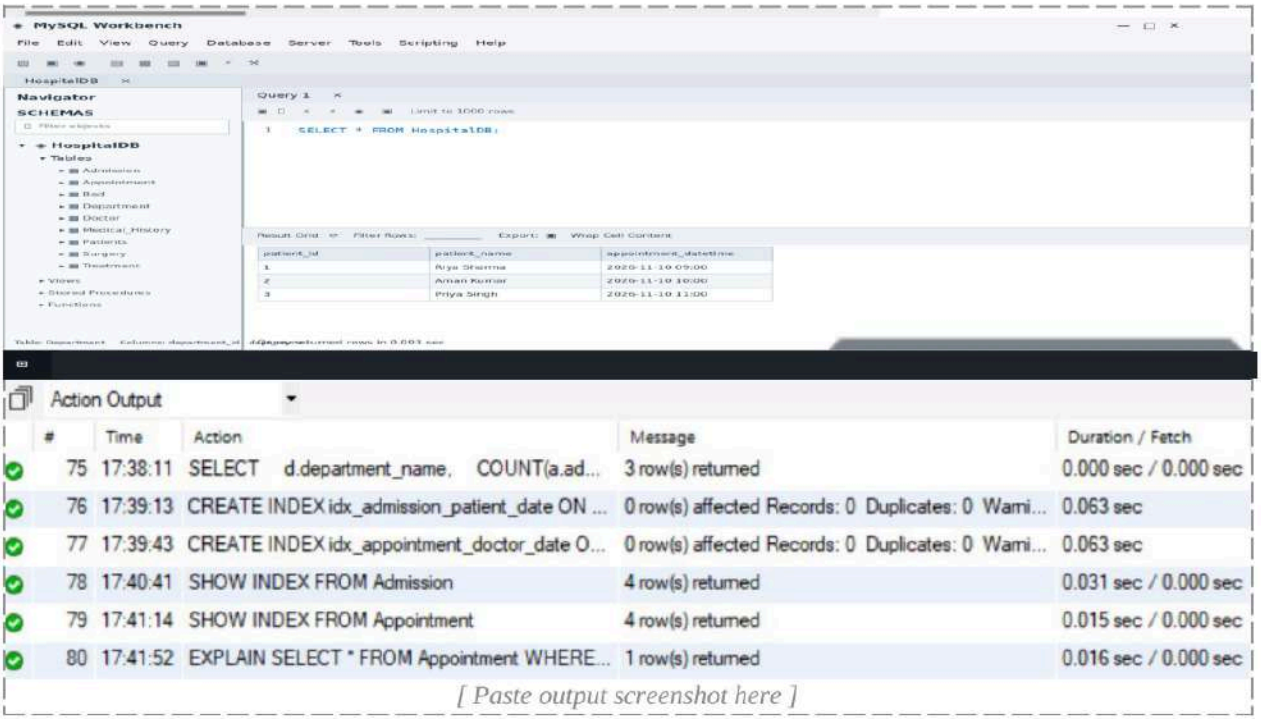
### 1. Objective

To analyse how MySQL executes a doctor-wise appointment query and whether it uses an index.

### 2. SQL Query

```
EXPLAIN
SELECT *
FROM Appointment
WHERE doctor_id = 1;
```

## 3. Execution Screenshot




*[ Paste output screenshot here ]*

## 4. Explanation

EXPLAIN shows the execution plan without running the query. The columns possible_keys and key show which index is considered and chosen, type shows the access method (ref means an index lookup, ALL means a full table scan) and rows shows the estimated number of rows examined. With an index on doctor_id the query reads only the matching rows instead of scanning the whole table. Because the sample table has only five rows, the optimizer may still decide that a full scan is cheaper; on a real hospital database with lakhs of appointments, the index lookup gives a large improvement. EXPLAIN provides an estimated execution plan, not a measured benchmark. The key and possible_keys columns help identify index selection, while type and rows provide clues about the access method and estimated work. With only five sample appointments, MySQL may prefer a table scan; meaningful performance comparison requires a much larger dataset and actual timing measurements.

# VIEWS AND ACCESS CONTROL

A view is a stored SELECT query that behaves like a virtual table. Views simplify complex joins and also provide security because users can be given access to a view without being given access to the base tables. Two views are created in this project.

## Query 48: Create and Display the Current Patients View

### 1. Objective

To create a view that always shows the patients who are currently admitted along with their admission date and bed number.

### 2. SQL Query

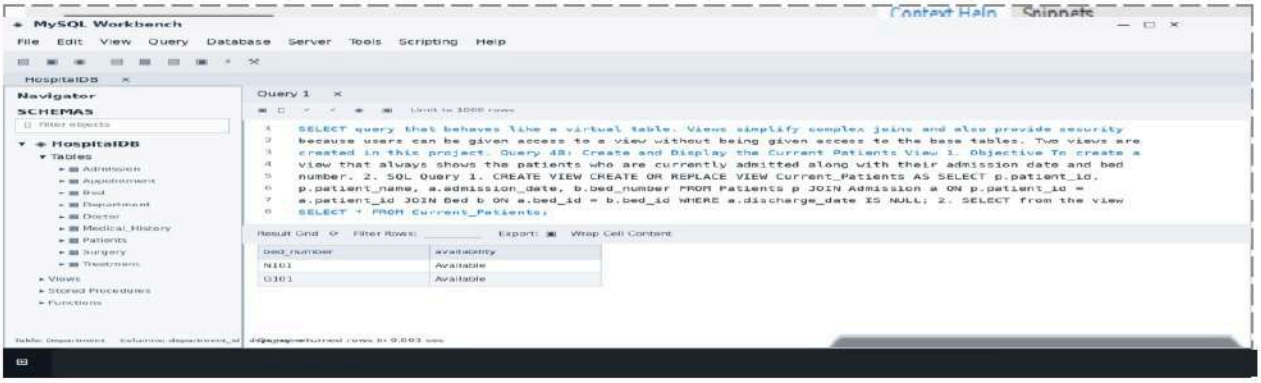
*1. CREATE VIEW*

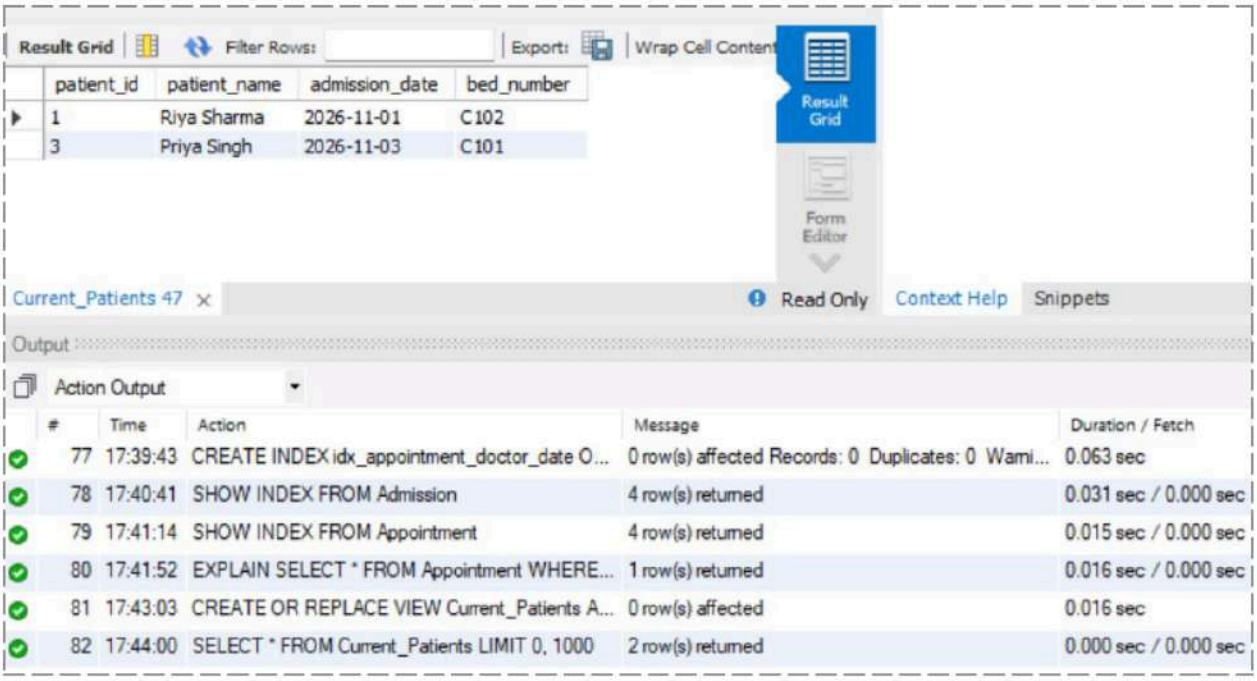
```
CREATE OR REPLACE VIEW Current_Patients AS
SELECT
    p.patient_id,
    p.patient_name,
    a.admission_date,
    b.bed_number
FROM Patients p
JOIN Admission a
ON p.patient_id = a.patient_id
JOIN Bed b
ON a.bed_id = b.bed_id
WHERE a.discharge_date IS NULL;
```

*2. SELECT from the view*

```
SELECT * FROM Current_Patients;
```

### 3. Execution Screenshot






### 4. Explanation

The view joins Patient, Admission and Bed and keeps only the rows with discharge_date IS NULL. Because the view is evaluated every time it is queried, it always reflects the latest admissions and discharges. CREATE OR REPLACE VIEW creates the view or updates it if it already exists. Querying the view returns Riya Sharma (C102) and Priya Singh (C101). A view stores the SELECT definition rather than a separate copy of the result rows. When the view is queried, it reflects the current underlying table data. This makes it convenient for ward staff because the same join and active-admission condition do not need to be rewritten in every report.

## Query 49: Create and Display the Front Desk View

### 1. Objective

To create a restricted view for front-desk staff that shows appointment information but hides the diagnosis and treatment details.

### 2. SQL Query

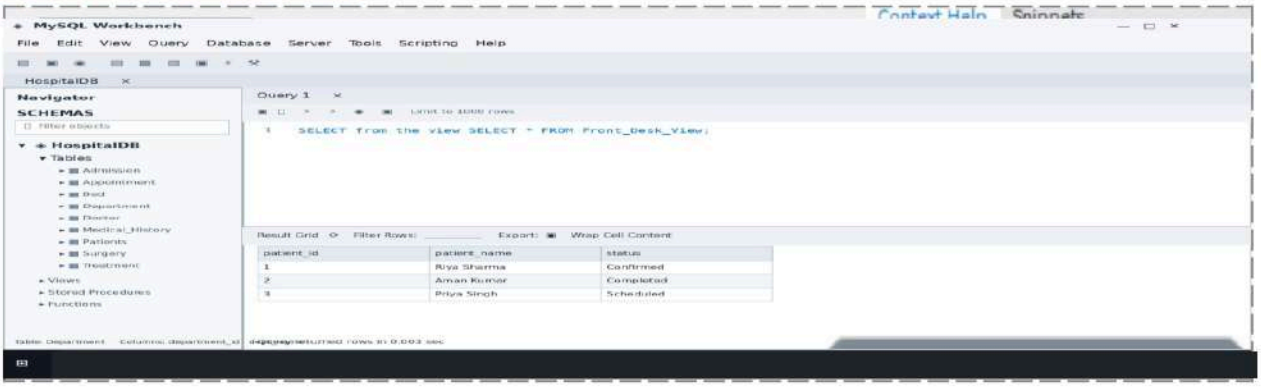
*1. CREATE VIEW*

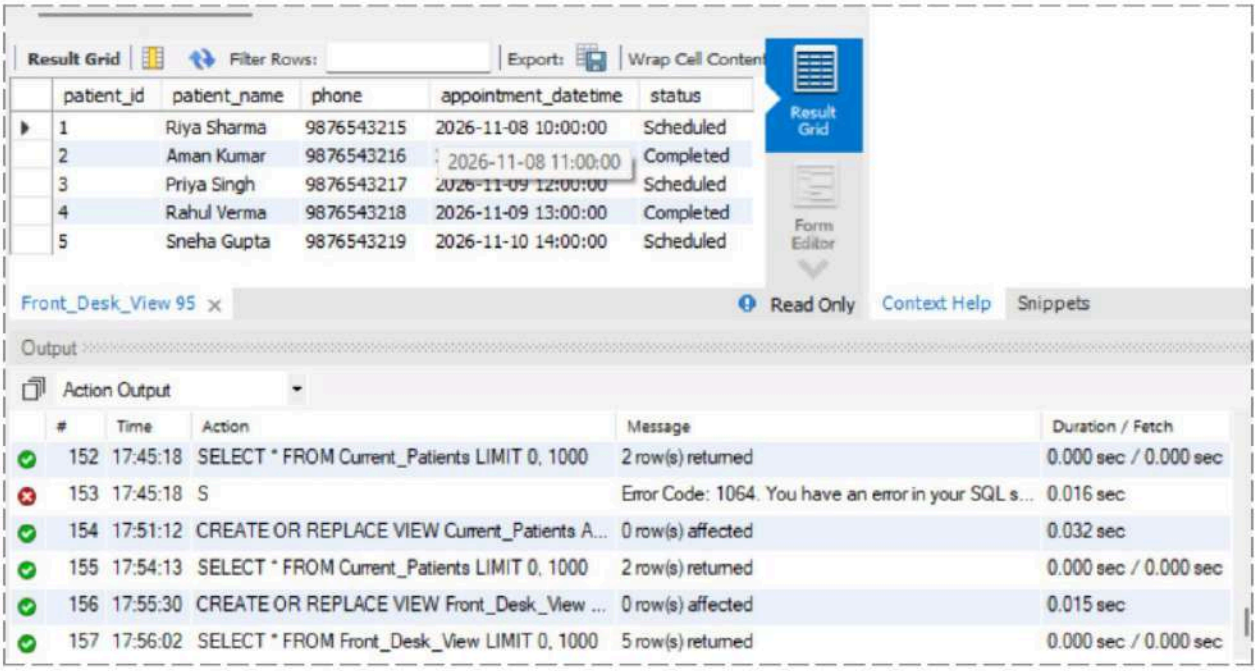
```
CREATE OR REPLACE VIEW Front_Desk_View AS
SELECT
    p.patient_id,
    p.patient_name,
    p.phone,
    a.appointment_datetime,
    a.status
FROM Patients p
JOIN Appointment a
ON p.patient_id = a.patient_id;
```

*2. SELECT from the view*

```
SELECT * FROM Front_Desk_View;
```

## 3. Execution Screenshot




## 4. Explanation

The view contains only the patient id, name, phone, appointment date-time and status. The columns of Medical_History (diagnosis) and Treatment (treatment_details) are not part of it, so the front-desk staff can manage appointments without seeing confidential medical information. Access can be controlled by granting the SELECT privilege on Front_Desk_View alone and not on the base tables. The view is a column-level restriction: it omits diagnosis and treatment fields from the result. For this to provide actual protection, the database administrator must grant front-desk users SELECT on the view and withhold direct SELECT access to the underlying sensitive tables. A view alone does not secure data if users still have unrestricted base-table permissions.

# WINDOW FUNCTIONS

Window functions perform calculations across a set of rows that are related to the current row, without collapsing the rows into groups as GROUP BY does. The functions DENSE_RANK(), AVG() OVER(), LAG() and LEAD() are used in this project.

## Query 50: Rank Doctors by Number of Appointments (DENSE_RANK)

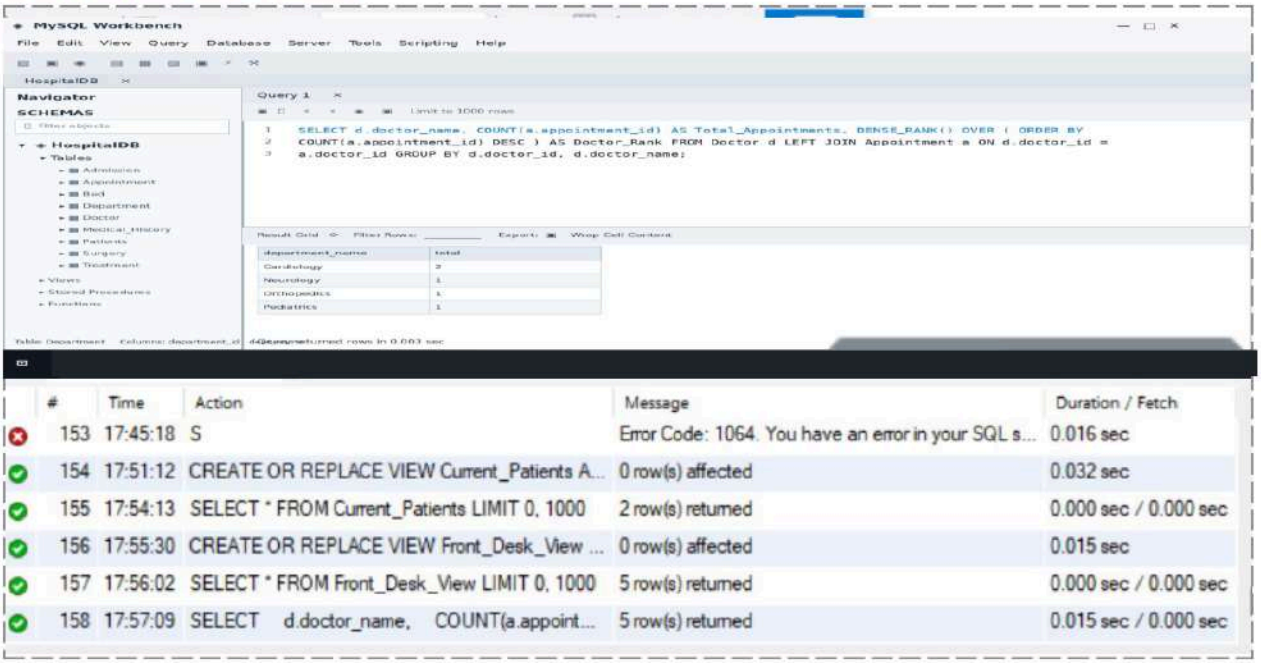
### 1. Objective

To rank the doctors according to the number of appointments they have.

### 2. SQL Query

```
SELECT
    d.doctor_name,
    COUNT(a.appointment_id) AS Total_Appointments,
    DENSE_RANK() OVER (
        ORDER BY COUNT(a.appointment_id) DESC
    ) AS Doctor_Rank
FROM Doctor d
LEFT JOIN Appointment a
ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.doctor_name;
```

### 3. Execution Screenshot




### 4. Explanation

The appointments of each doctor are counted first and DENSE_RANK() then ranks the doctors in descending order of that count. DENSE_RANK gives the same rank to equal counts and does not leave gaps in the ranking. As every doctor has one

appointment, all five doctors receive rank 1. The aggregate count is calculated for each doctor before DENSE_RANK assigns the rank. Doctors with equal counts receive the same rank, and the next distinct count receives the next consecutive rank. Because the sample gives every doctor one appointment, all five receive rank 1; a larger, uneven dataset would show meaningful differences.

## Query 51: Average Appointment Count (AVG OVER)

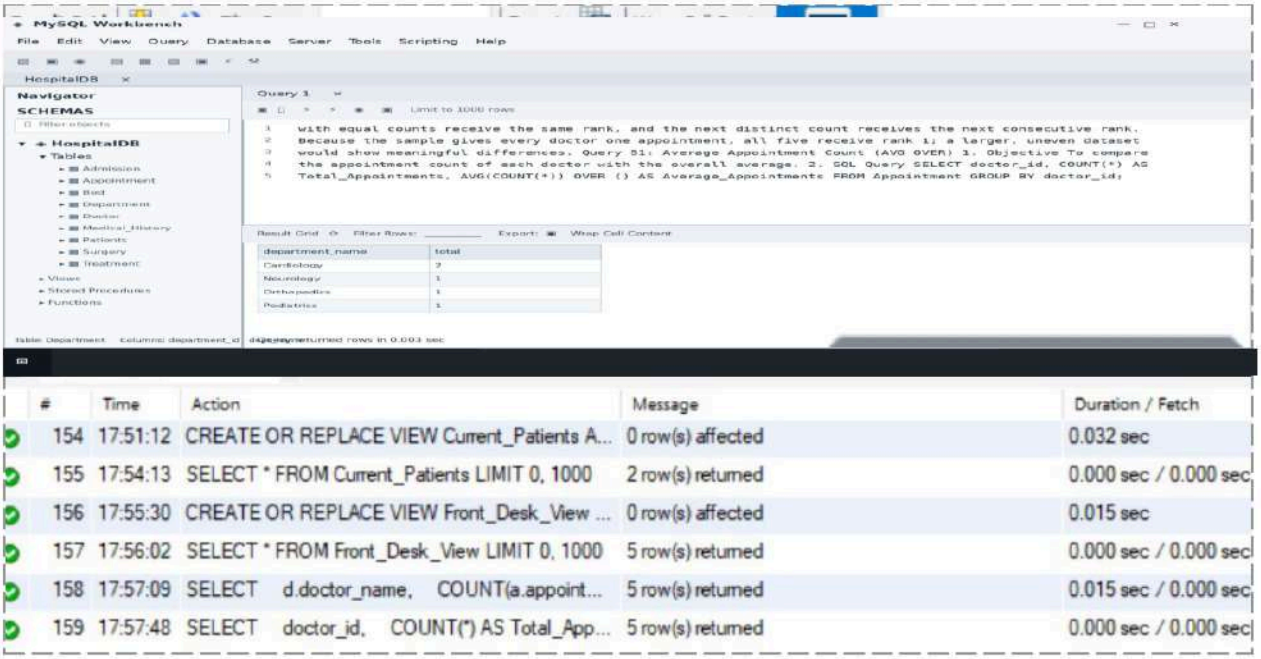
### 1. Objective

To compare the appointment count of each doctor with the overall average.

### 2. SQL Query

```
SELECT
    doctor_id,
    COUNT(*) AS Total_Appointments,
    AVG(COUNT(*)) OVER () AS Average_Appointments
FROM Appointment
GROUP BY doctor_id;
```

### 3. Execution Screenshot




### 4. Explanation

GROUP BY first gives the appointment count of every doctor, and AVG(COUNT(*)) OVER () then calculates the average of these counts over all rows and shows it in every row. The empty OVER () means that the whole result is one window. Each doctor can therefore be compared with the average (1.0 in the present data). GROUP BY produces one row per doctor, and AVG(COUNT(*)) OVER() calculates the average across those grouped rows. Unlike a normal aggregate without OVER, the window calculation keeps each doctor's row visible while repeating the overall average beside it. In the sample, each doctor has one appointment, so the average is 1.0.

## Query 52: Previous Appointment Using LAG

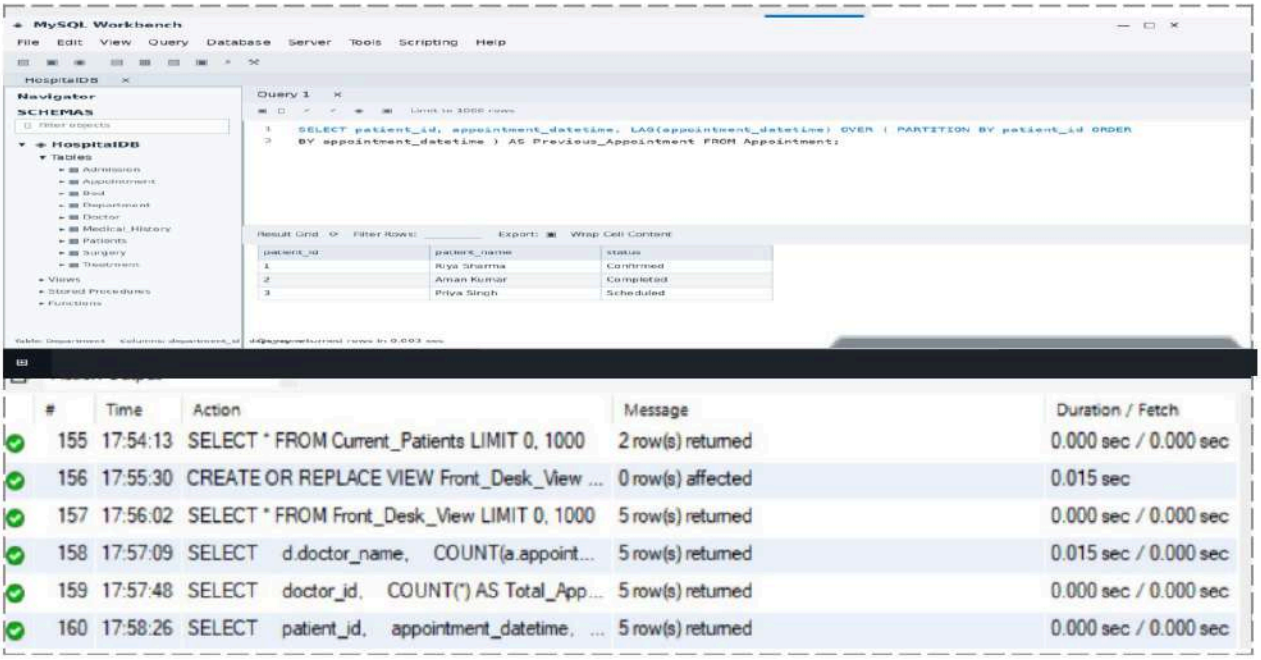
### 1. Objective

To display the previous appointment date of each patient.

### 2. SQL Query

```
SELECT
    patient_id,
    appointment_datetime,
    LAG(appointment_datetime) OVER (
        PARTITION BY patient_id
        ORDER BY appointment_datetime
    ) AS Previous_Appointment
FROM Appointment;
```

### 3. Execution Screenshot




### 4. Explanation

PARTITION BY patient_id creates a separate window for each patient and ORDER BY arranges the appointments by date. LAG() returns the value of the previous row in that window. Since every patient has only one appointment in the sample data, Previous_Appointment is NULL for all rows; it will show the earlier visit as soon as a patient has more than one appointment. LAG reads a value from an earlier row within the same patient's partition. PARTITION BY prevents one patient's appointment from being treated as the previous appointment of another patient, and ORDER BY defines the chronological sequence. The first row in each partition has no previous row, so its LAG value is NULL.

## Query 53: Next Appointment Using LEAD

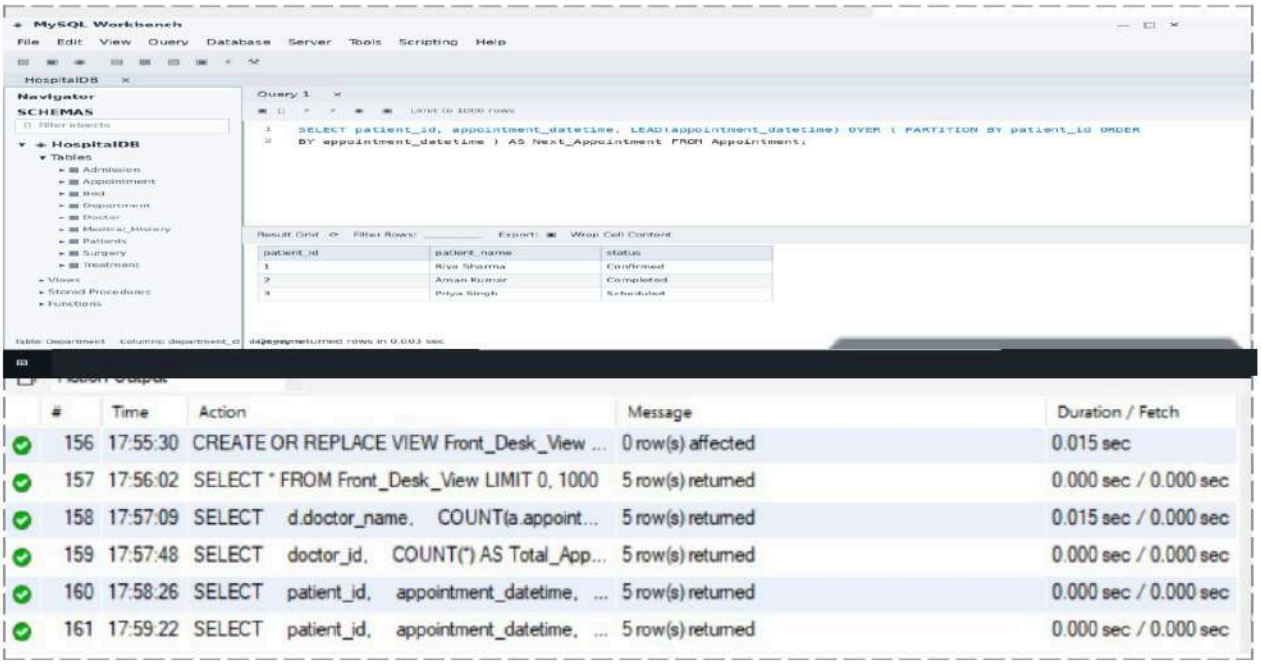
### 1. Objective

To display the next appointment date of each patient.

## 2. SQL Query

```
SELECT
    patient_id,
    appointment_datetime,
    LEAD(appointment_datetime) OVER (
        PARTITION BY patient_id
        ORDER BY appointment_datetime
    ) AS Next_Appointment
FROM Appointment;
```

## 3. Execution Screenshot




## 4. Explanation

LEAD() is the opposite of LAG(): it looks at the next row inside the patient's window and returns the next appointment date. It is useful for follow-up reminders. In the present data each patient has only one appointment, so Next_Appointment is NULL. LEAD reads a value from a later row in the ordered partition. It is useful for identifying the next planned visit or measuring the gap between appointments. Since the sample contains only one appointment for each patient, there is no following row and the result is NULL for every patient.

# RESULTS AND DISCUSSION

The Hospital Management database was created and tested with sample data. The results of the queries are summarised below.

- All nine tables were created successfully and the relationships between them worked correctly through foreign keys.
- The sample data contained 5 departments, 5 doctors, 5 patients, 5 beds, 5 appointments, 4 admissions, 5 history records, 5 treatments and 3 surgeries.
- The in-patient query showed that two patients (Riya Sharma and Priya Singh) are currently admitted, because their discharge date is NULL.
- The bed query showed two available beds (N101 and G101), matching the Bed INSERT statements. C101, C102 and O101 are marked Occupied in the sample data.
- The surgery schedule query listed the three surgeries in date order, from 12 to 14 November 2026.
- The department-wise admission query showed that Cardiology had the highest number of admissions (2).
- Subqueries and joins retrieved combined information from several tables, for example the appointments with patient and doctor names.
- Window functions ranked the doctors and calculated the average appointments. As each doctor and patient has only one appointment in the sample data, all doctors have rank 1 and LAG()/LEAD() return NULL; with more data they will show real ranking and appointment history.

**Index Trade-off**

Indexes on Admission(patient_id, admission_date) and Appointment(doctor_id, appointment_datetime) make searching by patient or doctor faster, because MySQL does not have to scan the whole table. However, every index must be updated each time a row is inserted, updated or deleted, and it also uses extra storage. On a table with very frequent inserts, too many indexes slow down writing. Therefore, indexes should be created only on the columns that are used often in WHERE, JOIN and ORDER BY clauses.

**Views and Security**

The Front_Desk_View shows only appointment information and hides diagnosis and treatment details. Staff can be given permission on the view instead of the base tables, which protects the confidential medical data of patients.

# CONCLUSION

This project designed a relational Hospital Management System database named HospitalDB. It contains nine tables for departments, doctors, patients, appointments, beds, admissions, medical history, treatments and surgeries. The tables are connected through primary and foreign keys so that records can be retrieved across the hospital workflow without repeatedly storing the same descriptive information. The design demonstrates how a database schema is planned around entities, attributes, identifiers and relationships.

The SQL implementation covers the full process from creating the database and tables to inserting sample data and generating reports. SELECT statements retrieve records, WHERE filters individual rows, JOIN combines related tables, and GROUP BY with aggregate functions produces summaries. Subqueries answer questions involving membership or averages. These queries demonstrate practical tasks such as finding current in-patients, checking bed availability, reviewing appointment status and preparing a surgery schedule. The results depend on the supplied sample records and should be interpreted in that context.

The project also demonstrates selected advanced DBMS features. Composite indexes are designed around patient-history and doctor-schedule lookups, and EXPLAIN is used to inspect the optimizer's estimated access plan. Views package common reports and can support least-privilege access when permissions are configured correctly. Window functions rank doctors and compare appointment counts while preserving row-level output. The small sample dataset is sufficient to show query behaviour, but it is not large enough to prove production-scale performance gains.

Working through the project strengthened my understanding of relational modelling, referential integrity, SQL query construction, indexing, views and analytical functions. The current version is an academic prototype, not a complete deployed hospital information system. Future improvements could include billing, pharmacy and medicine inventory, nurse and operating-theatre scheduling, role-based user accounts, audit logs, validation rules for bed allocation and surgery conflicts, backup and recovery procedures, and testing with a larger synthetic dataset. These additions would make the design closer to the needs of a real hospital while requiring careful attention to patient privacy and data security.

.

# REFERENCES

The following resources were used for understanding database concepts, SQL implementation, and MySQL operations during this project.


1. **MySQL Documentation –** Official documentation for SQL commands, database creation, table constraints, indexes, views, and window functions. https://dev.mysql.com/doc/

2. **MySQL 8.0 Reference Manual** – Documentation for SQL statements, joins, aggregate functions, and window functions.
https://dev.mysql.com/doc/refman/8.0/en/

3. **MySQL Workbench Documentation –** Reference for database development, SQL execution, and database management.
https://dev.mysql.com/doc/workbench/en/

4. **W3Schools SQL Tutorial** – Reference for SQL syntax, queries, constraints, and database operations. https://www.w3schools.com/sql/

5. **GeeksforGeeks – DBMS –** Reference for relational database concepts, normalization, keys, and indexing. https://www.geeksforgeeks.org/dbms/

6. **Draw.io (diagrams.net) –** Used for creating the Entity Relationship Diagram (ER Diagram). https://www.drawio.com/

## APPENDIX A: SQL EXECUTION OUTPUTS

The following output captures were generated by executing the project sample data and representative queries in the local SQLite engine available in the preparation environment. They are genuine query results, but they are NOT screenshots of MySQL Workbench and do not prove execution inside MySQL. MySQL-specific syntax (such as AUTO_INCREMENT, TIMESTAMPDIFF, SHOW INDEX and CREATE OR REPLACE VIEW) was translated to equivalent SQLite syntax for this compatibility check. Run the original SQL in MySQL Workbench to obtain native MySQL screenshots.

**Data check:** The Bed INSERT statements mark C101, C102 and O101 as Occupied, while N101 and G101 are Available. Therefore, the sample data contains two available beds, not three. The source report's results text was corrected to match the inserted records. The original curly quote around the C101 value was also corrected to a normal SQL quote so the INSERT statement can run.

### Table check

Executed query: SELECT name AS table_name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%' ORDER BY name;




### Record counts

Executed query: SELECT 'Department' AS table_name,COUNT(*) AS total FROM Department UNION ALL SELECT 'Doctor',COUNT(*) FROM Doctor UNION ALL SELECT 'Patients',COUNT(*) FROM Patients UNION ALL SELECT 'Bed',COUNT(*) FROM Bed UNION ALL SELECT 'Appointment',COUNT(*) FROM Appointment UNION ALL SELECT 'Admission',COUNT(*) FROM Admission UNION ALL SELECT 'Medical_History',COUNT(*) FROM Medical_History UNION ALL SELECT 'Treatment',COUNT(*) FROM Treatment UNION ALL SELECT 'Surgery',COUNT(*) FROM Surgery;




### Doctors and departments

Executed query: SELECT d.doctor_name,d.specialization,dp.department_name FROM Doctor d JOIN Department dp ON d.department_id=dp.department_id ORDER BY d.doctor_id;




### Currently admitted patients

Executed query: SELECT p.patient_name,a.admission_date,b.bed_number FROM Admission a JOIN Patients p ON a.patient_id=p.patient_id JOIN Bed b ON a.bed_id=b.bed_id WHERE a.discharge_date IS NULL ORDER BY a.admission_id;



## Available beds by department

Executed query: SELECT d.department_name,b.bed_number,b.availability FROM Bed b JOIN Department d ON b.department_id=d.department_id WHERE b.availability='Available' ORDER BY b.bed_id;




## Appointments by status

Executed query: SELECT status,COUNT(*) AS total FROM Appointment GROUP BY status ORDER BY status;



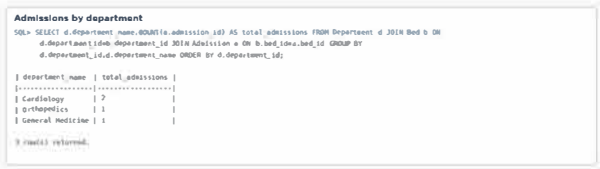

## Surgery schedule

Executed query: SELECT p.patient_name,d.doctor_name,s.surgery_date FROM Surgery s JOIN Patients p ON s.patient_id=p.patient_id JOIN Doctor d ON s.doctor_id=d.doctor_id ORDER BY s.surgery_date;



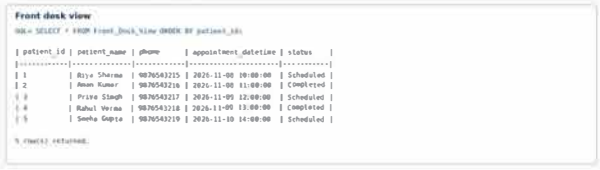

## Admissions by department

Executed query: SELECT d.department_name,COUNT(a.admission_id) AS total_admissions FROM Department d JOIN Bed b ON d.department_id=b.department_id JOIN Admission a ON b.bed_id=a.bed_id GROUP BY d.department_id,d.department_name ORDER BY d.department_id;



## Front desk view

Executed query: SELECT * FROM Front_Desk_View ORDER BY patient_id;




## Doctor ranking

Executed query: SELECT d.doctor_name,COUNT(a.appointment_id) AS total_appointments,DENSE_RANK() OVER(ORDER BY COUNT(a.appointment_id) DESC) AS doctor_rank FROM Doctor d LEFT JOIN Appointment a ON d.doctor_id=a.doctor_id GROUP BY d.doctor_id,d.doctor_name ORDER BY doctor_rank,d.doctor_name;




## Average appointments

Executed query: SELECT doctor_id,COUNT(*) AS total_appointments,ROUND(AVG(COUNT(*)) OVER(),2) AS average_appointments FROM Appointment GROUP BY doctor_id ORDER BY doctor_id;




## Previous and next appointments

Executed query: SELECT patient_id,appointment_datetime,LAG(appointment_datetime) OVER(PARTITION BY patient_id ORDER BY appointment_datetime) AS previous_appointment,LEAD(appointment_datetime) OVER(PARTITION BY patient_id ORDER BY appointment_datetime) AS next_appointment FROM Appointment ORDER BY patient_id;

